

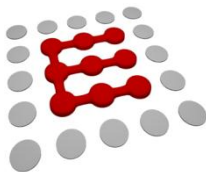


北京理工大学
Beijing Institute of Technology

本科实验报告

实验名称： 计算机原理与应用实验

课程名称：	计算机原理与应用	实验时间：	
任课教师：		实验地点：	
实验教师：		实验类型： ☐ 原理验证 ☒ 综合设计 ☐ 自主创新	
学生姓名：			
学号/班级：		组 号：	
学 院：	信息与电子学院	同组搭档：	
专 业：	电子信息工程(徐特立英才班)	成 绩：	



信息与电子学院

SCHOOL OF INFORMATION AND ELECTRONICS

目录

实验一 汇编语言编程基础实验 ()	3
1. 实验内容	3
2. 实验要求	3
3. 程序介绍	3
4. 程序流程图	3
5. 实验结果	3
6. 实验过程分析	4
实验一 汇编语言编程基础实验 ()	15
1. 实验内容	15
2. 实验要求	15
3. 程序介绍	15
4. 程序流程图	16
5. 实验结果	16
6. 实验过程分析	17
7. 心得体会	27
实验一 汇编语言编程基础实验 ()	29
1. 实验内容	29
2. 实验要求	29
3. 程序介绍	29
4. 程序流程图	29
5. 实验结果	30
6. 实验过程分析	30
实验一 汇编语言编程基础实验 ()	35
1. 实验目的	35
2. 实验要求	35
3. 实验内容	35
4. 实验流程图	35
5. 实验过程与结果分析	36
6. 实验心得	38

实验二 8255A 可编程并行接口实验.....	40
1. 实验目的	40
2. 实验原理和内容	40
3. 实验程序	43
4. 实验结果	47
5. 实验分析 ()	52
6. 实验分析 ()	54
7. 实验分析 ()	57
8. 实验分析 ()	57
实验三 使用 8259 中断实现定时/计数器实验.....	59
1. 实验目的	59
2. 实验原理和内容	59
3. 实验程序	62
4. 实验结果	71
5. 实验分析 ()	73
6. 实验分析 ()	75
7. 实验分析 ()	78
8. 实验分析 ()	80
实验四 使用 8251A 的串行接口应用实验.....	81
1. 实验目的	81
2. 实验原理和内容	81
3. 实验程序	84
4. 实验结果	87
5. 实验分析 ()	89
6. 实验分析 ()	91
7. 实验分析 ()	95
8. 实验分析 ()	96

实验一 汇编语言编程基础实验 ()

1. 实验内容

通过调试给定汇编源程序 EXP1.asm, 理解汇编语言源程序结构、变量定义、寻址方式和基本控制流程。在 EXP1.asm 的基础上, 编写 copy_data 子程序, 实现内存数据的定义和跨数据段的数据搬移功能。

2. 实验要求

- 1) 编写源程序;
- 2) 利用 debug 命令, 使用 L、U、G 及 DDS:0 命令调试并运行程序;
- 3) 详细记录每一步所用的命令, 查看结果的方法和具体结果, 体现内存中源地址数据搬移到目标地址的效果。

3. 程序介绍

- 待搬移的数据: DATA 数据段中 B_BUF3 变量存储的字符串 'COMPUTER', 包含 8 个 ASCII 字符。
- 搬移的源位置: DATA 段的 B_BUF3 变量偏移地址, 通过 lea si, B_BUF3 获取。
- 搬移的目标位置: EDATA 附加段的起始位置 ES:0。
- 数据搬移思路: 采用循环结构逐字节复制的方法。通过设置源指针 SI 指向 B_BUF3, 目标指针 DI 指向 EDATA 段起始位置, 计数器 CX 设置为 8, 然后使用循环结构逐个字节地将数据从源地址复制到目标地址。在复制过程中, 每次循环读取一个字节到 AL 寄存器, 再写入目标位置, 同时递增 SI 和 DI 指针, 递减 CX 计数器, 直到 CX 为 0 时完成所有数据的搬移。

4. 程序流程图

程序流程图如下图 4-1 所示:



图 4-1 整个程序流程图

5. 实验结果

在调试环境中, 首先输入命令 g0048, 使程序运行至断点地址 0048 处暂停执行。该地址对应 call disp_letter 指令, 因此程序在调用字母显示子程序前停止。

随后输入 des:0 命令，查看 EDATA 附加段的内存初始状态，可见该段内容全部由字符 s 填充，如图 5-1 所示：

```

-g0048
AX=46FD BX=0126 CX=0695 DX=0000 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0048  NU DN EI PL ZR NA PE NC
07C5:0048 8D161C02      LEA      DX,[021C1]          DS:021C=6C50
-des:0
0795:0000 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss

```

图 5-1 EDATA 内存初始状态

接着输入 g 命令，使程序继续执行。根据屏幕提示完成相应的输入输出操作后，再次输入 des:0 命令查看 EDATA 附加段的内存状态。此时可以观察到该段中已完整存储了源数据 B_BUF3 的内容，即字符串 COMPUTER 对应的 ASCII 码，如图 5-2 所示：

```

-g
Please input char : n000000
result           : n000000

Program terminated normally
-des:0
0795:0000 43 4F 4D 50 55 54 45 52-73 73 73 73 73 73 73 73 73 COMPUTERssssssss
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss

```

图 5-2 字符复制后 EDATA 内存状态

6. 实验过程分析

1. 寻址方式调试

寻址方式是汇编语言访问内存的核心手段，本次调试通过单步执行结合寄存器/内存查看，逐一验证了寄存器寻址、立即数寻址、寄存器间接寻址、直接寻址的有效性。

1) 段寄存器与栈保护配置

调试启动后，程序首先执行寄存器压栈保护与段寄存器初始化指令。通过 t 指令单步跟踪，观察到程序先将关键寄存器值压入栈中，随后将 DATA 段地址载入 AX 并赋值给 DS，将 EDATA 段地址载入 AX 并赋值给 ES，如图 6-1 所示：

```

AX=0000 BX=0000 CX=0610 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0760 ES=0760 SS=07B5 CS=07C5 IP=0004  NU UP EI PL ZR NA PE NC
07C5:0004 B87007      MOV     AX,0770
-t
AX=0770 BX=0000 CX=0610 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0760 ES=0760 SS=07B5 CS=07C5 IP=0007  NU UP EI PL ZR NA PE NC
07C5:0007 8ED8      MOV     DS,AX
-t
AX=0770 BX=0000 CX=0610 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0770 ES=0760 SS=07B5 CS=07C5 IP=0009  NU UP EI PL ZR NA PE NC
07C5:0009 B89507      MOV     AX,0795
-t
AX=0795 BX=0000 CX=0610 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0770 ES=0760 SS=07B5 CS=07C5 IP=000C  NU UP EI PL ZR NA PE NC
07C5:000C 8EC0      MOV     ES,AX

```

图 6-1 段寄存器初始化

通过 r 指令查看寄存器状态，如图 6-2 所示：

```

-r
AX=0795 BX=0000 CX=0610 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=000E  NU UP EI PL ZR NA PE NC

```

图 6-2 寄存器的值

DS=0770H、ES=0795H，与预设段地址完全一致，确保后续内存访问的地址基准正确。

2) 多类型寻址的分步验证

继续输入 t 指令执行 mov bx,cx 与 mov cl,al 指令，通过 r 指令核查，如图 6-3 所示：

```

-r
AX=0795 BX=0610 CX=0695 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0012  NU UP EI PL ZR NA PE NC
07C5:0012 B303      MOV     BL,03

```

图 6-3 寄存器的值

BX 值从初始 0000H 更新为 CX 的初始值 0610H，CL 值从初始 00H 更新为 AL 的初始值 95H，证明寄存器间数据传递有效。

继续输入 t 指令执行 mov bl,03 与 mov bp,3003H 指令，输入 r 指令显示 BL=03H、BP=3003H，与立即数完全匹配，验证了立即数直接赋值的正确性，如图 6-4 所示：

```

-r
AX=0795 BX=0603 CX=0695 DX=0000 SP=00FC BP=3003 SI=0000 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0017  NU UP EI PL ZR NA PE NC
07C5:0017 26      ES:
07C5:0018 C607FD      MOV     BYTE PTR [BX],FD      ES:0603=00

```

图 6-4 寄存器的值

继续输入 t 指令执行 mov byte ptr es:[bx],0FDH（对应-3 补码）与 mov word ptr es:[bx+1],0FFFDH（对应-3 补码）指令。输入 des:0603 指令查看附加数据段：ES:0603H 地址值为 0FDH，ES:0604H-0605H 地址值为 0FFFDH，与预期补码一

致，间接寻址写入功能生效，如图 6-5 所示：

```
-dds:0603
0795:0600      FD FD FF 00 00-00 00 00 00 00 00 00 00 00 .....
0795:0610 00 00 00 00 00 00 00 00-00 00 00 00 00 00 .....
0795:0620 00 00 00 00 00 00 00 00-00 00 00 00 00 00 .....
0795:0630 00 00 00 00 00 00 00 00-00 00 00 00 00 00 .....
0795:0640 00 00 00 00 00 00 00 00-00 00 00 00 00 00 .....
0795:0650 00 00 00 00 00 00 00 00-00 00 00 00 00 00 .....
0795:0660 00 00 00 00 00 00 00 00-00 00 00 00 00 00 .....
0795:0670 00 00 00 00 00 00 00 00-00 00 00 00 00 00 .....
0795:0680 00 00 00
```

图 6-5 附加数据段的值

继续输入 t 指令执行 mov byte ptr [0101H],0AH。输入 dds:100 指令查看数据段：DS:0101H 地址值从初始值更新为 0AH，直接寻址操作成功，如图 6-6 所示：

```
-dds:100
0770:0100 40 0A 0A FC FC 0A 19 00-00 00 43 4F 4D 50 55 54 0.....COMPUT
0770:0110 45 52 01 4A 41 4E 02 46-45 42 03 6D 61 72 31 32 ER.JAN.FEB.mar12
0770:0120 33 34 61 42 63 64 00 00-00 00 00 00 00 00 00 34aBcd.....
0770:0130 2A 2A 35 35 35 2A 2A 35-35 35 2A 2A 35 35 35 2A **555**555**555*
0770:0140 2A 35 35 35 2A 2A 35 35-35 FD FF FD FF 19 00 12 *555**555.....
0770:0150 01 30 00 01 00 02 00 03-00 04 00 05 00 3C 00 00 .0.....<..
0770:0160 00 00 00 00 00 00 00 00-00 69 01 00 00 00 00 43 .....i....C
0770:0170 50 00 00 D2 04 00 00 0C-00 00 00 22 00 00 00 12 P....."....
```

图 6-6 数据段的值

后续输入 t 指令执行“载入 B_BUF6 偏移地址”、“写入立即数 10000”、“写入 CS 值”“B_BUF2 第 1 个字节值送入 AL，W_BUF2 的字型值送入 AX”直接寻址指令，通过 r、dds:126 指令核查，均确认数据准确写入目标地址，如图 6-7 所示：

```
-r
AX=0795 BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=002D NU UP EI PL ZR NA PE NC
07C5:002D C7071027 MOV WORD PTR [BX+1,2710] DS:0126=0000
```

(a)

```
-dds:126
0770:0120      10 27-C5 07 00 00 00 00 00 .....
0770:0130 2A 2A 35 35 35 2A 2A 35-35 35 2A 2A 35 35 35 2A **555**555**555*
0770:0140 2A 35 35 35 2A 2A 35 35-35 FD FF FD FF 19 00 12 *555**555.....
0770:0150 01 30 00 01 00 02 00 03-00 04 00 05 00 3C 00 00 .0.....<..
0770:0160 00 00 00 00 00 00 00 00-00 69 01 00 00 00 00 43 .....i....C
0770:0170 50 00 00 D2 04 00 00 0C-00 00 00 22 00 00 00 12 P....."....
0770:0180 01 70 07 3A 00 00 00 00-00 00 00 00 00 00 32 .p.:.....Z
0770:0190 3C 00 00 00 00 00 00 34-12 00 00 00 00 00 00 00 <.....4.....
0770:01A0 00 00 00 00 00 00
```

(b)

```
-r
AX=070A BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0037 NU UP EI PL ZR NA PE NC
07C5:0037 A14B01 MOV AX,[014B] DS:014B=FFFD
```

(c)

```
-r
AX=FFFD BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=003A NU UP EI PL ZR NA PE NC
07C5:003A F8 CLC
```

(d)

图 6-7 寄存器与数据段的值

2. 标志寄存器调试

标志寄存器是汇编程序判断分支、控制流程的核心，本次调试重点验证标志位修改与栈保护/恢复功能。

1) 标志位初始配置与保护

程序先执行 `clc`、`std`、`pushf` 指令。通过 `r` 指令查看标志位：CF 对应符号为 NC，即 CF=0，DF 对应符号为 DN，即 DF=1，证明 `clc` 与 `std` 执行有效，`pushf` 指令将当前标志状态存入栈中，为后续恢复做准备，如图 6-8 所示：

```
-r
AX=FFFD BX=0126 CX=0695 DX=0000 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=003D NU DN EI PL ZR NA PE NC
07C5:003D 9F LAHF
```

图 6-8 标志寄存器的值

2) 标志位修改与核查

执行 `lahf` 指令，`r` 指令显示 AX 值从 FFFDH 更新为 46FDH，指令执行有效，如图 6-9 所示：

```
-r
AX=46FD BX=0126 CX=0695 DX=0000 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=003E NU DN EI PL ZR NA PE NC
07C5:003E F9 STC
```

图 6-9 寄存器的值

随后执行 `stc`、`cld` 指令，再次通过 `r` 指令核查：CF 符号变为 CY，即 CF=1，DF 符号变为 UP，即 DF=0，标志位修改符合预期，如图 6-10 所示：

```
-r
AX=46FD BX=0126 CX=0695 DX=0000 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0040 NU UP EI PL ZR NA PE CY
07C5:0040 9D POPF
```

图 6-10 标志寄存器的值

3) 标志位恢复验证

最后执行 `popf` 指令，通过 `r` 指令查看：标志位状态恢复为“NC+DN”，与 `pushf` 压栈前完全一致，证明栈操作对标志寄存器的保护与恢复功能有效，避免了后续操作对初始标志状态的破坏，如图 6-11 所示：

```
-r
AX=46FD BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0041 NU DN EI PL ZR NA PE NC
07C5:0041 E80400 CALL 0048
```

图 6-11 标志寄存器的值

3. disp_letter 子程序调试

`disp_letter` 子程序负责“提示输入-字符串接收-字母判断-结果输出”，调试需结合反汇编定位入口、断点控制流程、单步跟踪循环，确保每一步数据处理准确。

1) 子程序入口定位与断点设置

通过 U 指令对程序反汇编，如图 6-12 所示：

```
-u
07C5:0041 E80400      CALL    0048
07C5:0044 E85A00      CALL    00A1
07C5:0047 CB          RETF
07C5:0048 8D161C02      LEA     DX,[021C]
07C5:004C B409      MOV     AH,09
07C5:004E CD21      INT     21
07C5:0050 8D16B501      LEA     DX,[01B5]
07C5:0054 B40A      MOV     AH,0A
07C5:0056 CD21      INT     21
07C5:0058 8D164602      LEA     DX,[0246]
07C5:005C B409      MOV     AH,09
07C5:005E CD21      INT     21
07C5:0060 8D1EB501      LEA     BX,[01B5]
```

图 6-12 使用 U 指令查找地址

发现 call 0048 对应子程序入口地址 0048H，因此在 0048H 处设置断点，输入 g0048 命令，程序执行至该地址后停止，成功进入 disp_letter 子程序，如图 6-13 所示：

```
-g0048
AX=46FD BX=0126 CX=0695 DX=0000 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0048  NU DN EI PL ZR NA PE NC
07C5:0048 8D161C02      LEA     DX,[021C]          DS:021C=6C50
```

图 6-13 使用 g0048 指令

2) 提示信息输出与字符串输入验证

子程序前 3 条指令（lea dx,[021CH]、mov ah,09H、int 21H）负责输出提示信息“Please input char :”。通过 r 指令查看 DX=021CH，执行 int 21H 后界面成功显示提示，如图 6-14 所示：

```
-r
AX=09FD BX=0126 CX=0695 DX=021C SP=00F4 BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=F000 IP=14A0  NU DN DI PL ZR NA PE NC
F000:14A0 FB          STI
```

(a)

```
-t
Please input char :
AX=09FD BX=0126 CX=0695 DX=021C SP=00F4 BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=F000 IP=14A5  NU DN EI PL ZR NA PE NC
F000:14A5 CF          IRET
```

(b)

图 6-14 寄存器的值和输出提示信息

随后执行字符串输入指令（lea dx,[01B5H]、mov ah,0AH、int 21H），输入“cyb”后，通过 dds:01B5H 指令查看数据段：DS:01B5H-01B7H 地址依次存储“c、y、b”，字符串准确存入 char1 变量地址，如图 6-15 所示：

```

-t
cyb
AX=0AFD BX=0126 CX=0695 DX=01B5 SP=00F4 BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=F000 IP=14A5 NU DN EI PL ZR NA PE NC
F000:14A5 CF IRET
-dds:01b5
0770:01B0          64 03 63-79 62 0D 00 00 00 00 00 00 00 00 00 00 d.cyb.....
0770:01C0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:01D0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:01E0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:01F0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0200 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0210 00 00 00 00 00 00 00 00-00 00 00 00 50 6C 65 61 .....Please
0770:0220 73 65 20 69 6E 70 75 74-20 63 68 61 72 20 3A 20 se input char :
0770:0230 24 72 65 73 75 $resu

```

图 6-15 数据段的值

3) 提示信息输出与字符串输入验证

输入完成后，程序初始化 CX=0003H、BX=01B7H、DI=0000H，进入循环判断字母逻辑。由于“cyb”均为字母，程序不触发跳转，执行“字符复制-指针自增-计数器递减”流程：

- 单步执行 1 次循环后，dds:0 指令显示 DS:0000H=63H (“c”)，r 指令显示 BX=01B8H、DI=0001H、CX=0002H，符合预期，如图 6-16 所示：

```

-r
AX=0963 BX=01B8 CX=0003 DX=0246 SP=00FA BP=3003 SI=0126 DI=0001
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0083 NU DN EI PL NZ NA PE NC
07C5:0083 E2E8 LOOP 006D
-dds:0
0770:0000 63 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 c.....
0770:0010 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0020 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0030 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0040 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0050 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0060 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0070 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....

```

图 6-16 寄存器和数据段的值

- 3 次循环结束后，dds:0 指令显示 DS:0000H-0002H 依次存储“cyb”，CX=0000H，循环处理完成，如图 6-17 所示：

```

-t
AX=0962 BX=01BA CX=0001 DX=0246 SP=00FA BP=3003 SI=0126 DI=0003
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0083 NU DN EI PL NZ NA PD NC
07C5:0083 E2E8 LOOP 006D
-dds:0
0770:0000 63 79 62 00 00 00 00 00-00 00 00 00 00 00 00 00 cyb.....
0770:0010 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0020 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0030 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0040 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0050 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0060 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0070 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....

```

图 6-17 寄存器和数据段的值

- 最后执行“添加‘\$’结束符-输出结果”指令，界面显示“result:cyb”，子程序功能完全验证，如图 6-18 所示：

```

-t
result
AX=0924 BX=01BA CX=0000 DX=0231 SP=00F4 BP=3003 SI=0126 DI=0003
DS=0770 ES=0795 SS=07B5 CS=F000 IP=14A5 NU DN EI PL NZ NA PO NC
F000:14A5 CF IRET

-t
cyb
AX=0924 BX=01BA CX=0000 DX=0000 SP=00F4 BP=3003 SI=0126 DI=0003
DS=0770 ES=0795 SS=07B5 CS=F000 IP=14A5 NU DN EI PL NZ NA PO NC
F000:14A5 CF IRET

```

图 6-18 子程序输出

4. copy_data 子程序调试

1) 程序地址查找与断点设置

首先使用 U 命令逐行查找程序中的关键指令地址，如图 6-19 所示：

```

-u
07C5:0041 E80400 CALL 0048
07C5:0044 E85A00 CALL 00A1
07C5:0047 CB RETF
07C5:0048 8D161C02 LEA DX,[021C]
07C5:004C B409 MOV AH,09
07C5:004E CD21 INT 21
07C5:0050 8D16B501 LEA DX,[01B5]
07C5:0054 B40A MOV AH,0A
07C5:0056 CD21 INT 21
07C5:0058 8D164602 LEA DX,[0246]
07C5:005C B409 MOV AH,09
07C5:005E CD21 INT 21
07C5:0060 8D1EB501 LEA BX,[01B5]

```

图 6-19 使用 U 指令查找地址

通过反汇编查看代码，确定 00A1 为 call copy_data 命令行地址。

2) 初始状态检查

直接输入 g00a1 指令，程序执行到 call copy_data 命令行后停止，如图 6-20 所示：

```

-g00a1
Please input char : n000000
result : n000000

AX=0924 BX=01BE CX=0000 DX=0246 SP=00FA BP=3003 SI=0126 DI=0007
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00A1 NU DN EI PL NZ NA PE NC
07C5:00A1 50 PUSH AX

```

图 6-20 使用 g00a1 指令

此时程序暂停在数据复制子程序调用前。完成输入输出操作后，控制框自动展示寄存器值和下一步命令。

输入 des:0 指令查看 EDATA 附加段内容，如图 6-21 所示：

```

-des:0
0795:0000 73 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0010 73 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0020 73 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0030 73 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0040 73 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0050 73 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0060 73 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0070 73 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss

```

图 6-21 附加数据段的值

结果显示 EDATA 段全为 s，确认目标区域初始状态。

3) 子程序单步执行分析

使用 T 指令进行单步运行，执行 copy_data 子程序，如图 6-22 所示：

```
AX=0924 BX=01BE CX=0000 DX=0246 SP=00F8 BP=3003 SI=0126 DI=0007
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00A2 NU DN EI PL NZ NA PE NC
07C5:00A2 56          PUSH    SI
-t

AX=0924 BX=01BE CX=0000 DX=0246 SP=00F6 BP=3003 SI=0126 DI=0007
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00A3 NU DN EI PL NZ NA PE NC
07C5:00A3 57          PUSH    DI
-t

AX=0924 BX=01BE CX=0000 DX=0246 SP=00F4 BP=3003 SI=0126 DI=0007
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00A4 NU DN EI PL NZ NA PE NC
07C5:00A4 51          PUSH    CX
-t

AX=0924 BX=01BE CX=0000 DX=0246 SP=00F2 BP=3003 SI=0126 DI=0007
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00A5 NU DN EI PL NZ NA PE NC
07C5:00A5 1E          PUSH    DS
-t

AX=0924 BX=01BE CX=0000 DX=0246 SP=00F0 BP=3003 SI=0126 DI=0007
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00A6 NU DN EI PL NZ NA PE NC
07C5:00A6 06          PUSH    ES
```

图 6-22 使用 T 指令

观察到程序开始执行 PUSH 指令序列，依次将 AX、SI、DI、CX、DS、ES 寄存器压栈保护，IP 值变化显示程序正确进入子程序。

继续输入 T 指令，如图 6-23 所示：

```
-t
AX=0924 BX=01BE CX=0000 DX=0246 SP=00EE BP=3003 SI=0126 DI=0007
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00A7 NU DN EI PL NZ NA PE NC
07C5:00A7 BE0A01      MOV     SI,010A
-t

AX=0924 BX=01BE CX=0000 DX=0246 SP=00EE BP=3003 SI=010A DI=0007
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00AA NU DN EI PL NZ NA PE NC
07C5:00AA BF0000      MOV     DI,0000
-t

AX=0924 BX=01BE CX=0000 DX=0246 SP=00EE BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00AD NU DN EI PL NZ NA PE NC
07C5:00AD B90800      MOV     CX,0008
```

图 6-23 使用 T 指令

执行将 B_BUF3 的偏移地址赋值给 SI 源指针，设置目标偏移地址为 0，设置计数器为 8，准备复制 8 个字节。

4) 循环复制过程详细跟踪

输入 des:0 查看 EDATA 附加段内存，如图 6-24 所示：

```
-des:0
0795:0000 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssss
```

图 6-24 附加数据段的值

内存中全为 s，数据尚未复制。

继续输入 T 指令，如图 6-25 所示：

```
-t
AX=0924 BX=01BE CX=0008 DX=0246 SP=00EE BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B0 NU DN EI PL NZ NA PE NC
07C5:00B0 8A04 MOV AL,[SI] DS:010A=43
```

图 6-25 使用 T 指令

继续输入 T 指令，将 SI 指向的字节 C 读入 AL 寄存器，如图 6-26 所示：

```
-t
AX=0943 BX=01BE CX=0008 DX=0246 SP=00EE BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B2 NU DN EI PL NZ NA PE NC
07C5:00B2 26 ES:
07C5:00B3 8B05 MOV [DI],AL ES:0000=73
```

图 6-26 使用 T 指令

继续输入 T 指令，将 AL 中的 C 写入 ES:DI 指向的目标位置，如图 6-27 所示：

```
-t
AX=0943 BX=01BE CX=0008 DX=0246 SP=00EE BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B5 NU DN EI PL NZ NA PE NC
07C5:00B5 46 INC SI
```

图 6-27 使用 T 指令

继续输入 T 指令，源指针 SI 自增 1，指向下一个字节 O，如图 6-28 所示：

```
-t
AX=0943 BX=01BE CX=0008 DX=0246 SP=00EE BP=3003 SI=010B DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B6 NU DN EI PL NZ NA PO NC
07C5:00B6 47 INC DI
```

图 6-28 使用 T 指令

继续输入 T 指令，目标指针 DI 自增 1，指向下一个目标位置，如图 6-29 所示：

```
-t
AX=0943 BX=01BE CX=0008 DX=0246 SP=00EE BP=3003 SI=010B DI=0001
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B7 NU DN EI PL NZ NA PO NC
07C5:00B7 E2F7 LOOP 00B0
```

图 6-29 使用 T 指令

继续输入 T 指令，CX 自减 1 变为 7，由于 $CX \neq 0$ ，跳转回循环开始。此时寄存器状态：SI=B_BUF3+1；DI=0001H；CX=0007H。

输入 des:0 查看内存变化，如图 6-30 所示：

```

-des:0
0795:0000 43 73 73 73 73 73 73 73-73 73 73 73 73 73 73 C s s s s s s s s s s s s
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s

```

图 6-30 附加数据段的值

ES:0 位置已变为 C，其余位置仍为 s。

重复上述 T 指令序列，观察到：AL 读取 O；ES:1 位置写入 4FH；SI 增至 B_BUF3+2，DI 增至 0002H，CX 减至 0006H。

内存检查显示 ES:0-1 位置为 43H、4FH，确认前两个字符已复制，如图 6-31 所示：

```

-des:0
0795:0000 43 4F 73 73 73 73 73 73-73 73 73 73 73 73 73 C O s s s s s s s s s s s s
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s

```

图 6-31 附加数据段的值

继续使用 T 指令重复执行循环，每次循环完成一个字符的复制：

- 第 3 次循环：复制 M 到 ES:2，CX=0005H
- 第 4 次循环：复制 P 到 ES:3，CX=0004H
- 第 5 次循环：复制 U 到 ES:4，CX=0003H
- 第 6 次循环：复制 T 到 ES:5，CX=0002H
- 第 7 次循环：复制 E 到 ES:6，CX=0001H
- 第 8 次循环：复制 R 到 ES:7，CX=0000H

复制完成后，输入 des:0 指令查看 EDATA 附加段内存，如图 6-32 所示：

```

-des:0
0795:0000 43 4F 4D 50 55 54 45 52-73 73 73 73 73 73 73 C O M P U T E R s s s s s s s s
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s

```

图 6-32 附加数据段的值

显示 ES:0-7 位置依次为：43H、4FH、4DH、50H、55H、54H、45H、52H，对应 COMPUTER 字符串，确认数据搬移成功完成。

5) 复制完成与恢复现场

当 CX=0000H 时，LOOP 指令不再跳转，继续执行后续指令，如图 6-33 所示：

AX=0952	BX=01BE	CX=0000	DX=0246	SP=00EE	BP=3003	SI=0112	DI=0008
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00B9	NU	DN	EI
PL	NZ	NA	PO	NC			
07C5:00B9	07	POP	ES				
-t							
AX=0952	BX=01BE	CX=0000	DX=0246	SP=00F0	BP=3003	SI=0112	DI=0008
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00BA	NU	DN	EI
PL	NZ	NA	PO	NC			
07C5:00BA	1F	POP	DS				
-t							
AX=0952	BX=01BE	CX=0000	DX=0246	SP=00F2	BP=3003	SI=0112	DI=0008
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00BB	NU	DN	EI
PL	NZ	NA	PO	NC			
07C5:00BB	59	POP	CX				
-t							
AX=0952	BX=01BE	CX=0000	DX=0246	SP=00F4	BP=3003	SI=0112	DI=0008
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00BC	NU	DN	EI
PL	NZ	NA	PO	NC			
07C5:00BC	5F	POP	DI				
-t							
AX=0952	BX=01BE	CX=0000	DX=0246	SP=00F6	BP=3003	SI=0112	DI=0007
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00BD	NU	DN	EI
PL	NZ	NA	PO	NC			
07C5:00BD	5E	POP	SI				

图 6-33 使用 T 指令

执行恢复 ES 寄存器，恢复 DS 寄存器，恢复 CX 寄存器，恢复 DI 寄存器，恢复 SI 寄存器，恢复 AX 寄存器。

实验一 汇编语言编程基础实验 ()

1. 实验内容

通过调试给定汇编源程序 EXP1.asm, 理解汇编语言源程序结构、变量定义、寻址方式和基本控制流程。在 EXP1.asm 的基础上, 编写 copy_data 子程序, 实现内存数据的定义和跨数据段的数据搬移功能。

2. 实验要求

- 1) 编写源程序;
- 2) 利用 debug 命令, 使用 L、U、G 及 DDS:0 命令调试并运行程序;
- 3) 详细记录每一步所用的命令, 查看结果的方法和具体结果, 体现内存中源地址数据搬移到目标地址的效果。

3. 程序介绍

在汇编源程序 EXP1.asm 中, 首先在数据段定义了一些变量, 并存放相应的数据和字符。然后在堆栈段定义了 256 个 '#' 字符, 在附加数据段定义了 200H 个 's' 字符。在代码段中首先使用立即数寻址、寄存器寻址, 直接寻址, 寄存器间接寻址等方式进行寻址, 修改指定寄存器和内存空间中的值。然后是标志寄存器操作, 在这里使用了相关指令实现了对标志寄存器特定标志位的置 1 和清 0, 并使用堆栈操作实现标志寄存器的保护。接下来时显示英文字母子程序 disp_letter, 在子程序中首先输出提示信息 message1, 然后将字符串输入内存相应位置 char1, 接下来循环判断输入的字符串每个字符是否是英文字母并将其复制到内存中相应位置, 最后再将其输出。

在程序的最后需要编写 copy_data 子程序, 编写 copy_data 子程序如下所示:

```
copy_data  proc near
            mov ax, DATA
            mov ds, ax
            mov ax, EDATA
            mov es, ax          ; 初始化段寄存器

            mov si, offset B_BUF3 ; 初始化寄存器, si 为 B_BUF3 地址
            xor di, di          ; di 清 0
```



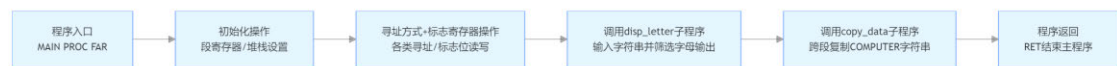
```

mov cx, 8          ; 设定循环次数为 8
COPY_LOOP:
    mov al, ds:[si]    ; 逐字节循环复制
    mov es:[di], al
    inc si
    inc di
    loop COPY_LOOP
    ret
copy_data endp

```

在上述的汇编程序中，需要搬移的数据为 DATA 数据段中 B_BUF3 变量所存储的字符串 'COMPUTER'，该字符串包含 8 个字符。搬移的源地址为 DATA 数据段中 B_BUF3 变量所在地址，该地址可通过指令 `mov si, offset B_BUF3` 进行获取，搬移的目标位置为 EDATA 附加段的起始位置 ES:0。在本程序中数据搬移采用逐字节循环复制的方式，具体操作如下：先将源变址指针 SI 指向 B_BUF3 所在地址，目标变址指针 DI 指向 EDATA 附加数据段的起始地址，同时将计数器 CX 的值设为 8 代表循环 8 次。随后进入循环流程，每次循环中先从源地址读取一个字节存入 AL 寄存器，再将 AL 中的字节写入目标地址，之后分别递增 SI 和 DI 指针以指向后续地址，同时 CX 计数器会随着循环的进行而递减。此循环过程会持续执行，直到 CX 计数器的值变为 0，此时 8 个字节的数据便全部完成从源地址到目标地址的搬移。

4. 程序流程图



5. 实验结果

在最开始的寻址方式程序中，程序代码可实现正确的数据搬移，具体可见实验过程分析部分。

在 `disp_letter` 子程序中，程序先输出提示信息，用户输入一串字符串，然后程序再将这个字符串输出，如下图所示：

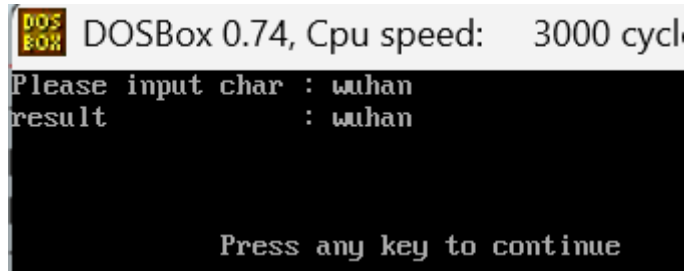


图 1: disp_letter 子程序运行结果

最后是 copy_data 子程序，首先点击调试按钮，在调试页面输入 g00a1 命令，程序会首先运行到字符串输入指令前，待用户输入字符串后运行到 copy_data 子程序入口地址处，此时附加数据段 EDATA 仍为初始状态，如下图所示：

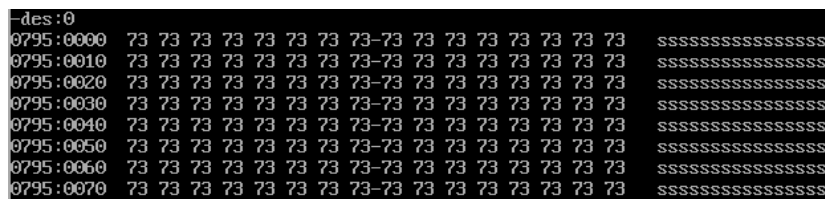


图 2: 附加数据段 EDATA 的内容

接下来输入 g 命令使程序运行结束，输入 des:0 命令查看 EDATA 附加数据段的内存状态，如下图所示：

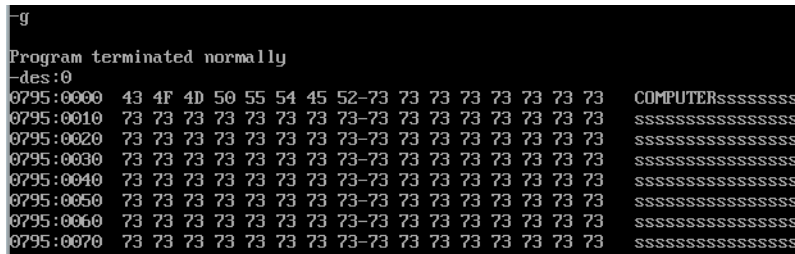


图 3: 程序运行结果

可以观察到源地址中的字符串'COMPUTER'已经被复制到附加数据段，这与实验要求相符，也说明 copy_data 子程序编写正确。

6. 实验过程分析

1. 寻址方式调试

首先点击调试按钮，进入调试界面后输入 t 指令单步执行程序。程序的开始首先压栈保护寄存器，然后初始化段寄存器 DS 和 ES，如下图所示：

```

-t
AX=0000 BX=0000 CX=060D DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0760 ES=0760 SS=07B5 CS=07C5 IP=0004  NU UP EI PL ZR NA PE NC
07C5:0004 B87007      MOV     AX,0770
-t
AX=0770 BX=0000 CX=060D DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0760 ES=0760 SS=07B5 CS=07C5 IP=0007  NU UP EI PL ZR NA PE NC
07C5:0007 8ED8      MOV     DS,AX
-t
AX=0770 BX=0000 CX=060D DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0770 ES=0760 SS=07B5 CS=07C5 IP=0009  NU UP EI PL ZR NA PE NC
07C5:0009 B89507      MOV     AX,0795
-t
AX=0795 BX=0000 CX=060D DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0770 ES=0760 SS=07B5 CS=07C5 IP=000C  NU UP EI PL ZR NA PE NC
07C5:000C 8EC0      MOV     ES,AX

```

图 4：段寄存器初始化

继续输入 t 指令单步执行程序，接下来是寻址方式示例代码。前两行语句为寄存器寻址方式，将 CX 寄存器的值送入 BX，AL 寄存器的值送入 CL，此时输入 r 指令查看寄存器的值如下图所示：

```

-r
AX=0795 BX=060D CX=0695 DX=0000 SP=00FC BP=0000 SI=0000 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0012  NU UP EI PL ZR NA PE NC
07C5:0012 B303      MOV     BL,03

```

图 5：查看寄存器的值

可见 BX 和 CL 中的数值已发生相应变化。下面的两条语句为立即数寻址方式，分别将立即数 3 送入 BL，立即数 3003H 送入 BP，此时输入 r 指令查看寄存器的值如下图所示：

```

-r
AX=0795 BX=0603 CX=0695 DX=0000 SP=00FC BP=3003 SI=0000 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0017  NU UP EI PL ZR NA PE NC
07C5:0017 26      ES:
07C5:001B C607FD      MOV     BYTE PTR [BX],FD      ES:0603=00

```

图 6：查看寄存器的值

可见寄存器 BL 和 BP 的数值已发生相应变化。下面的两条语句为寄存器间接寻址方式，分别将字节型立即数-3 送入附加数据段 BX 偏移处，字型立即数-3 送入附加数据段 BX+1 偏移处，此时输入 des:0603 指令查看附加数据段对应地址处的值如下图所示：

```

-des:0603
0795:0600      FD FD FF 00 00-00 00 00 00 00 00 00 00 00 00 00 .....
0795:0610 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0795:0620 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0795:0630 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0795:0640 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0795:0650 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0795:0660 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0795:0670 00 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0795:0680 00 00 00 .....

```

图 7：查看附加数据段的值

可见 BX 和 BX+1 偏移处的值分别为 FD 和 FFD，这为-3 对应的补码表示，

这说明数据被成功搬移。下面一条语句是直接寻址方式，将立即数 0AH（换行符）送入 B_BUF1 的第 2 个字节，此时输入 dds:0 指令查看附加数据段对应地址处的值如下图所示：

```
-dds:100
0770:0100 40 0A 0A FC FC 0A 19 00-00 00 43 4F 4D 50 55 54 0.....COMPUT
0770:0110 45 52 01 4A 41 4E 02 46-45 42 03 6D 61 72 31 32 ER.JAN.FEB.mar12
0770:0120 33 34 61 42 63 64 00 00-00 00 00 00 00 00 00 34aBcd.....
0770:0130 2A 2A 35 35 35 2A 2A 35-35 35 2A 2A 35 35 35 2A **555**555**555*
0770:0140 2A 35 35 35 2A 2A 35 35-35 FD FF FD FF 19 00 12 *555**555.....
0770:0150 01 30 00 01 00 02 00 03-00 04 00 05 00 3C 00 00 .0.....<..
0770:0160 00 00 00 00 00 00 00 00-00 69 01 00 00 00 43 .....i.....C
0770:0170 50 00 00 D2 04 00 00 0C-00 00 00 22 00 00 12 P....."....
```

图 8：查看数据段的值

可见数据段第 2 个字节的值已变为 0A，表明搬移成功。接下来的两条语句分别将 B_BUF6 的偏移地址送入 BX，B_BUF6 的偏移地址送入 SI，此时输入 r 指令查看寄存器的值如下图所示：

```
-r
AX=0795 BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=002D NU UP EI PL ZR NA PE NC
07C5:002D C7071027 MOV WORD PTR [BX],2710 DS:0126=0000
```

图 9：查看寄存器的值

可见 B_BUF6 的偏移地址已被送入 BX 和 SI 中，并且经计算可得送入地址与实际地址相符。下面的两条语句分别将字型立即数 10000 送入 BX 指向的内存单元，将 CS 送入 BX+2 指向的内存单元，此时输入 dds:126 指令查看数据段对应地址处的值如下图所示：

```
AX=0795 BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0034 NU UP EI PL ZR NA PE NC
07C5:0034 A00201 MOV AL,[0102] DS:0102=0A
-dds:126
0770:0120 10 27-C5 07 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0770:0130 2A 2A 35 35 35 2A 2A 35-35 35 2A 2A 35 35 35 2A **555**555**555*
0770:0140 2A 35 35 35 2A 2A 35 35-35 FD FF FD FF 19 00 12 *555**555.....
0770:0150 01 30 00 01 00 02 00 03-00 04 00 05 00 3C 00 00 .0.....<..
0770:0160 00 00 00 00 00 00 00 00-00 69 01 00 00 00 43 .....i.....C
0770:0170 50 00 00 D2 04 00 00 0C-00 00 00 22 00 00 12 P....."....
0770:0180 01 70 07 3A 00 00 00 00-00 00 00 00 00 00 32 .p.....2
0770:0190 3C 00 00 00 00 00 00 34-12 00 00 00 00 00 00 00 <.....4.....
0770:01A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

图 10：查看数据段的值

可见 BX 和 BX+2 偏移处的值分别为立即数 10000 和段寄存器 CS 的值，表明搬移成功。寻址方式示例中最后两条语句为直接寻址方式，分别将 B_BUF2 的第 1 个字节值送入 AL，W_BUF2 的字型值送入 AX，此时输入 r 指令查看寄存器的值分别如下图所示：

```
-r
AX=070A BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0037 NU UP EI PL ZR NA PE NC
07C5:0037 A14B01 MOV AX,[014B] DS:014B=FFFF
```

(a)

```

-r
AX=FFFD BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=003A NU UP EI PL ZR NA PE NC
07C5:003A F8 CLC

```

(b)

图 11: 查看寄存器的值

可见寄存器 AX 的值随着搬移发生了相应变化。

2. 标志寄存器调试

在寻址方式示例代码结束后，下面是标志寄存器操作示例代码。前三行语句分别实现清进位标志 CF，置方向标志 DF 并将标志寄存器压栈保护，此时输入 r 指令查看寄存器的值如下图所示：

```

-r
AX=FFFD BX=0126 CX=0695 DX=0000 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=003D NU DN EI PL ZR NA PE NC
07C5:003D 9F LAHF

```

图 12: 查看标志寄存器

此时表征 CF 和 DF 的符号分别为 NC 和 DN，表明 CF 被清 0，DF 被置 1，上述语句操作有效。接下来将标志寄存器低 8 位送到 AH 寄存器，此时输入 r 指令查看寄存器的值如下图所示：

```

-r
AX=46FD BX=0126 CX=0695 DX=0000 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=003E NU DN EI PL ZR NA PE NC
07C5:003E F9 STC

```

图 13: 查看寄存器的值

可见 AX 寄存器的值已发生相应变化。下面两条语句分别实现置进位标志 CF 和清方向标志 DF 的功能，此时输入 r 指令查看寄存器的值如下图所示：

```

-r
AX=46FD BX=0126 CX=0695 DX=0000 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0040 NU UP EI PL ZR NA PE CY
07C5:0040 9D POPF

```

图 14: 查看标志寄存器

此时表征 CF 和 DF 的符号分别为 CY 和 UP，表明 CF 被置 1，DF 被清 0，上述语句操作有效。最后将原先压栈保护的标志寄存器弹出堆栈，此时输入 r 指令查看寄存器的值如下图所示：

```

-r
AX=46FD BX=0126 CX=0695 DX=0000 SP=00FC BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0041 NU DN EI PL ZR NA PE NC
07C5:0041 E80400 CALL 0048

```

图 15: 查看标志寄存器

此时的标志寄存器与图 相同，表明通过压栈和退栈确实起到了保护寄存器的作用。

3. disp_letter 子程序调试

首先点击调试按钮，进入调试页面后使用 U 命令查询汇编程序中每条指令地址并进行反汇编，如下图所示：

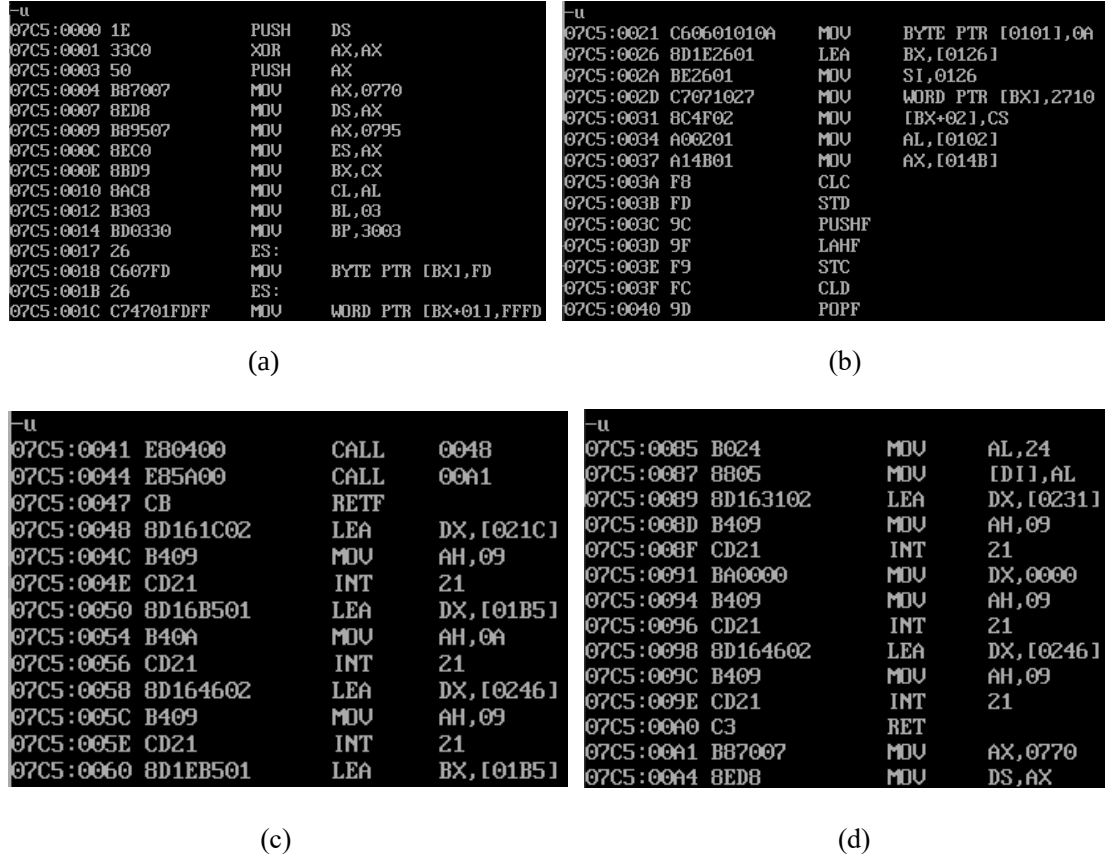


图 16: U 命令执行结果

观察上述反汇编的结果，可以发现地址 0048 和地址 00A1 存放的指令分别为 `lea dx,message1` 和 `mov ax, DATA`，即分别为子程序 `disp_letter` 和 `copy_data` 入口地址。因此上述反汇编后的 `CALL 0048` 代表 `call disp_letter` 语句，`CALL 00A1` 代表 `call copy_data` 语句。为了对子程序 `disp_letter` 进行调试，需要在子程序 `copy_data` 入口地址打一个断点，按照上述分析断点地址设置为 0048 即可。在调试界面中输入 `g0048` 命令，程序执行到子程序 `copy_data` 入口处停止，如下图所示：

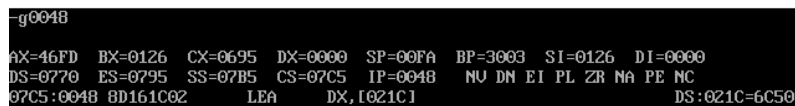


图 17: g00a1 命令执行结果

上图的结果表明已进入子程序 `disp_letter`。为了使输入输出调试方便，在 `disp_letter` 子程序可以使用 `p` 指令单步运行。前三行语句输出提示输入字符，将 `message1` 存放的字符串输出。首先将 `message1` 首地址存放在 `DX` 寄存器，由上述的反汇编操作可知 `message1` 首地址为 `021C`，分别输入 `r` 指令查看寄存器的值并且输入 `dds:0` 查看数据段的值，如下图所示：

```
-p
AX=09FD BX=0126 CX=0695 DX=021C SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=004E NU DN EI PL ZR NA PE NC
07C5:004E CD21 INT 21
```

(a)

```
-dds:021C
0770:0210                                     50 6C 65 61          Plea
0770:0220 73 65 20 69 6E 70 75 74-20 63 68 61 72 20 3A 20 se input char :
0770:0230 24 72 65 73 75 6C 74 20-20 20 20 20 20 20 20 20 $result
0770:0240 20 20 20 3A 20 24 0A 0D-24 00 00 00 00 00 00 00 : $.$.
0770:0250 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 sssssssssssssss
0770:0260 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 sssssssssssssss
0770:0270 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 sssssssssssssss
0770:0280 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 sssssssssssssss
0770:0290 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 sssssssssss
```

(b)

图 18: (a)查看寄存器的值; (b) 查看数据段的值

可见字符串 `message1` 首地址存储于 `DX` 寄存器中，并且字符串末尾有结束符 `'$'`。接下来输出字符串 `message1`，如下图所示：

```
-p
Please input char :
AX=09FD BX=0126 CX=0695 DX=021C SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0050 NU DN EI PL ZR NA PE NC
07C5:0050 8D16B501 LEA DX,[01B5] DS:01B5=0064
```

图 19: 输出提示输入字符

下面三行语句实现字符串输入到 `DS:DX` 所在地址，即 `char1` 变量所在地址，如下图所示：

```
-p
wuhan
AX=0AFD BX=0126 CX=0695 DX=01B5 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0058 NU DN EI PL ZR NA PE NC
07C5:0058 8D164602 LEA DX,[0246] DS:0246=0D0A
```

图 20: 输入字符串

由上述的反汇编操作可知 `DX` 存放的地址为 `01B5`，在完成字符串输入后在调试界面输入 `dds:01b5`，如下图所示：

```
-dds:01b5
0770:01B0                                     64 05 77-75 68 61 6E 0D 00 00 00 d.wuhan....
0770:01C0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0770:01D0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0770:01E0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0770:01F0 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0770:0200 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
0770:0210 00 00 00 00 00 00 00 00-00 00 00 50 6C 65 61 .....Plea
0770:0220 73 65 20 69 6E 70 75 74-20 63 68 61 72 20 3A 20 se input char :
0770:0230 24 72 65 73 75                                     $resu
```

图 21：查看数据段的值

可见字符串已输入到内存的指定位置。接下来输出回车换行，并初始化计数器 CX，使之成为 char1 的长度，并初始化寄存器 BX 和 DI，使之分别指向 char1 变量和数据段的开始位置，为接下来的循环输出做好准备，此时输入 r 指令查看寄存器的值如下图所示：

```
-r
AX=09FD BX=01B7 CX=0005 DX=0246 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=006D NU DN EI PL NZ NA PE NC
07C5:006D 8A07 MOV AL,[BX] DS:01B7=77
```

图 22：查看寄存器的值

在进入循环后要使用 t 命令调试，否则程序会跳过循环。由于先前输入的字符串第一个字符是英文字母，不满足程序的跳转条件，程序一直运行到 is_char 标志处。

```
AX=0977 BX=01B7 CX=0005 DX=0246 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0073 NU DN EI PL NZ NA PE NC
07C5:0073 3C7A CMP AL,7A
-t
AX=0977 BX=01B7 CX=0005 DX=0246 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0075 NU DN EI NG NZ AC PO CY
07C5:0075 770B JA 0082
-t
AX=0977 BX=01B7 CX=0005 DX=0246 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0077 NU DN EI NG NZ AC PO CY
07C5:0077 3C61 CMP AL,61
-t
AX=0977 BX=01B7 CX=0005 DX=0246 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0079 NU DN EI PL NZ NA PO NC
07C5:0079 7304 JNB 007F
-t
AX=0977 BX=01B7 CX=0005 DX=0246 SP=00FA BP=3003 SI=0126 DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=007F NU DN EI PL NZ NA PO NC
07C5:007F 8B05 MOV [DI],AL DS:0000=00
```

图 23：单步运行程序

继续运行程序，程序会将字符串第一个字符存入 DI 指向的偏移地址，并且寄存器 BX 和 DI 均自增 1 以指向下一个字符。完成上述操作后输入 r 指令查看寄存器的值并且输入 dds:0 查看数据段的值，如下图所示：

```
-r
AX=0977 BX=01B8 CX=0005 DX=0246 SP=00FA BP=3003 SI=0126 DI=0001
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0083 NU DN EI PL NZ NA PE NC
07C5:0083 E2E8 LOOP 006D
```

(a)

```
-dds:0
0770:0000 77 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 w.....
0770:0010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0770:0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0770:0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0770:0040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0770:0050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0770:0060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0770:0070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

(b)

图 24: (a)查看寄存器的值; (b) 查看数据段的值

可见此时寄存器 BX 和 DI 相较于图 均自增 1, char1 第一个字符'w'存入数据段的开始位置。最后到达 LOOP 指令使寄存器 CX 值减 1, 当 CX≠0 时程序跳转至 AGAIN 位置处再次开始循环。当 5 次循环结束后再次输入 r 指令查看寄存器的值并且输入 dds:0 查看数据段的值, 如下图所示:

```
-r
AX=096E BX=01BC CX=0000 DX=0246 SP=00FA BP=3003 SI=0126 DI=0005
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0085 NU DN EI PL NZ NA PO NC
07C5:0085 B024 MOV AL,24
```

(a)

```
-dds:0
0770:0000 77 75 68 61 6E 00 00 00-00 00 00 00 00 00 00 00 wuhan.....
0770:0010 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0020 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0030 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0040 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0050 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0060 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
0770:0070 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
```

(b)

图 25: (a)查看寄存器的值; (b) 查看数据段的值

此时字符串 char1 以全部存入数据段的开始位置, 寄存器 BX 和 DI 相较于图 均自增 5, CX 已减至 0。此时需要在字符串 char1 后加上'\$', 使后续输出可以正常结束。子程序的最后按照最开始的方法依次输出字符串 message2 和 char1, 并输出回车换行, 如下图所示:

```
-p
result :
AX=0924 BX=01BC CX=0000 DX=0231 SP=00FA BP=3003 SI=0126 DI=0005
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0091 NU DN EI PL NZ NA PO NC
07C5:0091 BA0000 MOV DX,0000
-p
AX=0924 BX=01BC CX=0000 DX=0000 SP=00FA BP=3003 SI=0126 DI=0005
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0094 NU DN EI PL NZ NA PO NC
07C5:0094 B409 MOV AH,09
-p
AX=0924 BX=01BC CX=0000 DX=0000 SP=00FA BP=3003 SI=0126 DI=0005
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0096 NU DN EI PL NZ NA PO NC
07C5:0096 CD21 INT 21
-p
wuhan
AX=0924 BX=01BC CX=0000 DX=0000 SP=00FA BP=3003 SI=0126 DI=0005
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=0098 NU DN EI PL NZ NA PO NC
07C5:0098 8D164602 LEA DX,[0246] DS:0246=0D0A
```

图 26: 子程序输出

4. copy_data 子程序调试

子程序 copy_data 按照上述程序结果中的内容进行编写。为了对子程序 copy_data 进行调试, 并观察程序运行过程中寄存器的变化, 需要在子程序 copy_data 入口地址打一个断点, 按照上述分析断点地址设置为 00A1 即可。在调


```

-t
AX=0743 BX=01BC CX=0008 DX=0246 SP=00FA BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B5 NU DN EI PL ZR NA PE NC
07C5:00B5 26      ES:
07C5:00B6 8805      MOV     DI,AL          ES:0000=73

```

(b)

```

-t
AX=0743 BX=01BC CX=0008 DX=0246 SP=00FA BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B8 NU DN EI PL ZR NA PE NC
07C5:00B8 46      INC     SI

```

(c)

```

-t
AX=0743 BX=01BC CX=0008 DX=0246 SP=00FA BP=3003 SI=010B DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B9 NU DN EI PL NZ NA PO NC
07C5:00B9 47      INC     DI

```

(d)

图 30: 子程序循环周期

图 7(a)(b)(c)(d)以此展示了 `copy_data` 子程序在一个循环周期内的执行步骤。首先将 `DS:SI` 指向的字节移入 `AL` 寄存器，接下来将 `AL` 寄存器中的字符移入 `ES:DI` 中，完成了字符向附加数据段 `EDATA` 的复制。再复制完成后，`SI` 和 `DI` 分别加 1，指向下一个字节。在一个循环周期执行结束后输入 `t` 指令，如下图所示：

```

-t
AX=0743 BX=01BC CX=0008 DX=0246 SP=00FA BP=3003 SI=010B DI=0001
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00BA NU DN EI PL NZ NA PO NC
07C5:00BA E2F7      LOOP    00B3

```

图 31: 循环周期的结束

`LOOP` 指令执行结束后，循环计数器 `CX` 减 1，当 `CX≠0` 时程序跳转回循环开始位置。当第一个循环周期结束后，寄存器 `SI` 和 `DI` 相较于循环开始时均自增 1，`CX` 减少至 7。输入 `des:0` 命令查看附加数据段 `EDATA` 的内容，如下图所示：

```

-des:0
0795:0000 43 73 73 73 73 73 73 73-73 73 73 73 73 73 73 C s s s s s s s s s s s s
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 s s s s s s s s s s s s

```

图 32: 查看附加数据段的值

在此时 `ES:0` 位置已变为 `'C'`，这说明第一个字符已成功复制，其余位置仍为 `'s'`。重复输入 `t` 指令单步运行程序，在第 2 个循环周期结束后，输入 `des:0` 命令查看附加数据段 `EDATA` 的内容，如下图所示：

本方法，更培养了耐心和细致的思维习惯，学会从底层硬件视角理解程序的执行逻辑。这不仅夯实了我对计算机原理基础知识的理解，更让我意识到汇编语言编程的核心在于对程序逻辑和细节的把控，为后续专业课程的学习积累了宝贵的实践经验。

实验一 汇编语言编程基础实验 ()

1. 实验内容

通过调试给定汇编源程序 EXP1.asm, 理解汇编语言源程序结构、变量定义、寻址方式和基本控制流程。在 EXP1.asm 的基础上, 编写 copy_data 子程序, 实现内存数据的定义和跨数据段的数据搬移功能。

2. 实验要求

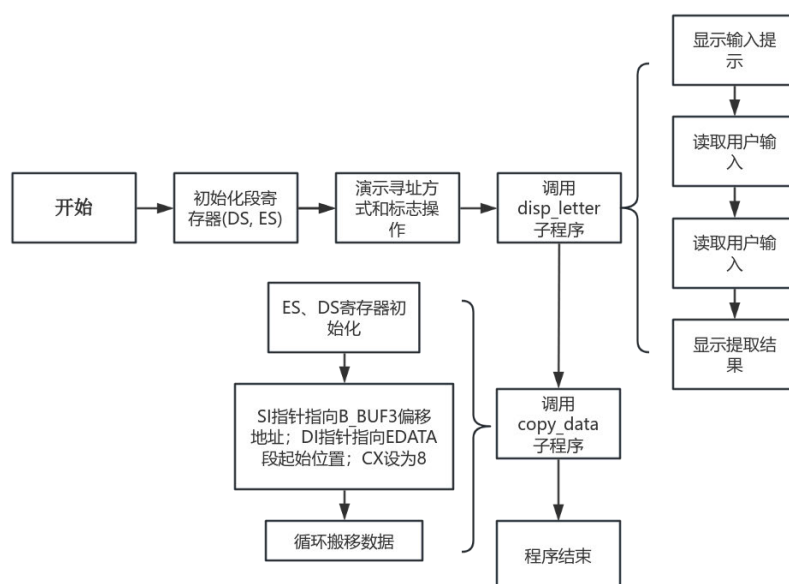
- 1) 编写源程序;
- 2) 利用 debug 命令, 使用 L、U、G 及 DDS:0 命令调试并运行程序;
- 3) 详细记录每一步所用的命令, 查看结果的方法和具体结果, 体现内存中源地址数据搬移到目标地址的效果。

3. 程序介绍

此程序计划将 DATA 数据段定义的 8 字节类型变量 B_BUF3 中的内容。其内容为 'COMPUTER' 字符串, 由 8 个 ASCII 码组成。数据源地址为 DATA 数据段中 B_BUF3 定义的位置, 可通过 lea si, B_BUF3 得到其偏移地址, 后续可用其进行下一步操作; 而目标地址则设置为 EDATA 附加段的起始位置 ES:0。

该程序通过设置源指针 SI、目标指针 DI 和计数器 CX, 同时利用 REP MOVSB 指令来实现高效的数据块搬移。采用正向复制方式(CLD), 确保数据按顺序正确复制。

4. 程序流程图




```

-u
07C5:0021 C60601010A    MOV     BYTE PTR [0101],0A
07C5:0026 8D1E2601    LEA     BX,[0126]
07C5:002A BE2601    MOV     SI,0126
07C5:002D C7071027    MOV     WORD PTR [BX],2710
07C5:0031 8C4F02    MOV     [BX+02],CS
07C5:0034 A00201    MOV     AL,[0102]
07C5:0037 A14B01    MOV     AX,[014B]
07C5:003A F8    CLC
07C5:003B FD    STD
07C5:003C 9C    PUSHF
07C5:003D 9F    LAHF
07C5:003E F9    STC
07C5:003F FC    CLD
07C5:0040 9D    POPF

```

```

-u
07C5:0041 E80400    CALL    0048
07C5:0044 E85A00    CALL    00A1
07C5:0047 CB    RETF
07C5:0048 8D161C02    LEA     DX,[021C]
07C5:004C B409    MOV     AH,09
07C5:004E CD21    INT     21
07C5:0050 8D16B501    LEA     DX,[01B5]
07C5:0054 B40A    MOV     AH,0A
07C5:0056 CD21    INT     21
07C5:0058 8D164602    LEA     DX,[0246]
07C5:005C B409    MOV     AH,09
07C5:005E CD21    INT     21
07C5:0060 8D1EB501    LEA     BX,[01B5]

```

图 6-1 使用 U 指令查找地址

其中 CALL 0048 代表 call disp_letter 命令行；CALL 00A1 代表 call copy_data 命令行。则可以知道 0048 为 call disp_letter 命令行地址；00A1 为 call copy_data 命令行地址。

直接输入 g00a1 指令，此时程序执行到 call copy_data 命令行后停止，完成输入输出操作后，控制框中自动展示寄存器的值和下一步命令行；此时输入 des:0 指令，可以查看 EDATA 附加段中的内容，如图 6-2 所示：

```

-g00a1
Please input char : jgk
result          : jgk

AX=0924 BX=01BA CX=0000 DX=0246 SP=00FA BP=3003 SI=0126 DI=0003
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00A1 NU DN EI PL NZ NA PO NC
07C5:00A1 B87007    MOV     AX,0770
-des:0
0795:0000 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 73 sssssssssssssss

```

图 6-2 再输入 g00a1 指令后的调试结果

而后使用 T 指令进行单步运行，执行下一步命令并展示寄存器的变化，如图 6-3 所示，该阶段完成了寄存器 DS 和 ES 的赋值（之前已经赋值过了，此处为了保险起见再进行一次赋值）。IP 值的变化也佐证了程序正在正确运行中。

AX=0770	BX=01BA	CX=0000	DX=0246	SP=00FA	BP=3003	SI=0126	DI=0003
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00A4	NU	DN	EI PL NZ NA PO NC
07C5:00A4	8ED8	MOV	DS,AX				
-t							
AX=0770	BX=01BA	CX=0000	DX=0246	SP=00FA	BP=3003	SI=0126	DI=0003
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00A6	NU	DN	EI PL NZ NA PO NC
07C5:00A6	B89507	MOV	AX,0795				
-t							
AX=0795	BX=01BA	CX=0000	DX=0246	SP=00FA	BP=3003	SI=0126	DI=0003
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00A9	NU	DN	EI PL NZ NA PO NC
07C5:00A9	8EC0	MOV	ES,AX				
-t							
AX=0795	BX=01BA	CX=0000	DX=0246	SP=00FA	BP=3003	SI=0126	DI=0003
DS=0770	ES=0795	SS=07B5	CS=07C5	IP=00AB	NU	DN	EI PL NZ NA PO NC
07C5:00AB	8D360A01	LEA	SI,[010A]				DS:010A=4F43

图 6-3 t 指令准备阶段的调试结果

继续输入 T 指令，将 B_BUF3 的偏移地址赋值给 SI，并设置目标偏移地址为 0，即 DI 为 0，同时赋值 CX 为 8，寄存器变化如图 6-4 所示：

```
AX=0795 BX=01BA CX=0000 DX=0246 SP=00FA BP=3003 SI=0126 DI=0003
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00AB NU DN EI PL NZ NA PO NC
07C5:00AB 8D360A01 LEA SI,[010A] DS:010A=4F43
-t

AX=0795 BX=01BA CX=0000 DX=0246 SP=00FA BP=3003 SI=010A DI=0003
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00AF NU DN EI PL NZ NA PO NC
07C5:00AF BF0000 MOV DI,0000
-t

AX=0795 BX=01BA CX=0000 DX=0246 SP=00FA BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B2 NU DN EI PL NZ NA PO NC
07C5:00B2 B90800 MOV CX,0008
-t

AX=0795 BX=01BA CX=0008 DX=0246 SP=00FA BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B5 NU DN EI PL NZ NA PO NC
07C5:00B5 FC CLD
```

图 6-4 赋值 SI、DI、CX 后寄存器的变化

执行下一步前，输入 `des:0` 查看 EDATA 附加段的内存，此时内存中还全是 's'(73H)，如图 6-5 所示：

```
-t
AX=0795 BX=01BA CX=0008 DX=0246 SP=00FA BP=3003 SI=010A DI=0000
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B6 NU UP EI PL NZ NA PO NC
07C5:00B6 F3 REPZ
07C5:00B7 A4 MUSB
-des:0
0795:0000 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 ssssssssssssssss
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 ssssssssssssssss
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 ssssssssssssssss
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 ssssssssssssssss
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 ssssssssssssssss
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 ssssssssssssssss
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 ssssssssssssssss
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 ssssssssssssssss
```

图 6-5 执行 rep movsb 命令前的 EDATA 附加内存

然后继续输入 T 指令，观察寄存器的变化，可以发现 SI、DI 自增了 1，而 CX 自减了 1，如图 6-6 所示。这是因为 `rep movsb` 命令行相当于是一个循环如图 6-7 所示，此处使用 `rep movsb` 命令能有效精简代码。

```

-t
AX=0795 BX=01BA CX=0007 DX=0246 SP=00FA BP=3003 SI=010B DI=0001
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B6 NU UP EI PL NZ NA PO NC
07C5:00B6 F3 REPZ
07C5:00B7 A4 MOUSB

```

图 6-6 寄存器变化

```

move_byte:
    MOV AL, [SI]      ; 从源地址读取一个字节到AL
    MOV ES:[DI], AL   ; 将AL写入目标地址
    INC SI            ; 源指针+1
    INC DI            ; 目标指针+1
    DEC CX            ; 计数器-1
    JNZ move_byte     ; 如果CX≠0, 继续循环

```

图 6-7 rep movsb 命令行等效循环

再次输入 des:0 查看 EDATA 附加段的内存，可以发现 ES:0 处内存已经存入数据'C'(43H)，如图 6-8 所示：

```

-des:0
0795:0000 43 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 C000000000000000
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000

```

图 6-8 ES:0 内存的变化

同上，输入 T 指令后再次查看寄存器和 EDATA 附加段的内存，均发生变化，SI、DI 自增了 1，CX 自减了 1，ES:1 处内存存入数据'O'(4FH)，如图 6-9 所示：

```

-t
AX=0795 BX=01BA CX=0006 DX=0246 SP=00FA BP=3003 SI=010C DI=0002
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B6 NU UP EI PL NZ NA PO NC
07C5:00B6 F3 REPZ
07C5:00B7 A4 MOUSB
-des:0
0795:0000 43 4F 73 73 73 73 73 73-73 73 73 73 73 73 73 73 C000000000000000
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 0000000000000000

```

图 6-9 寄存器和 ES:1 内存的变化

之后继续重复输入 T 指令，直至 CX 为 0，结束数据搬移过程。此时再输入 des:0 指令查看 EDATA 附加段的内存，如图 6-10 所示：

```

-t
AX=0795 BX=01B8 CX=0000 DX=0246 SP=00FA BP=3003 SI=0112 DI=0008
DS=0770 ES=0795 SS=07B5 CS=07C5 IP=00B8 NU UP EI PL NZ NA PO NC
07C5:00B8 C3 RET
-des:0
0795:0000 43 4F 4D 50 55 54 45 52-73 73 73 73 73 73 73 73 73 COMPUTERssssssss
0795:0010 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssssss
0795:0020 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssssss
0795:0030 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssssss
0795:0040 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssssss
0795:0050 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssssss
0795:0060 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssssss
0795:0070 73 73 73 73 73 73 73 73-73 73 73 73 73 73 73 73 sssssssssssssssss

```

图 6-10 完成数据的搬移

可见此时 B_BUF3 中的内容 'COMPUTER' 字符串已经成功搬移至 EDATA 附加段中。程序运行正常，达到试验目的。

实验一 汇编语言编程基础实验 ()

1. 实验目的

通过调试给定汇编源程序 EXP1.asm, 理解汇编语言源程序结构、变量定义、寻址方式和基本控制流程。在 EXP1.asm 的基础上, 编写 copy_data 子程序, 实现内存数据的定义和跨数据段的数据搬移功能。

2. 实验要求

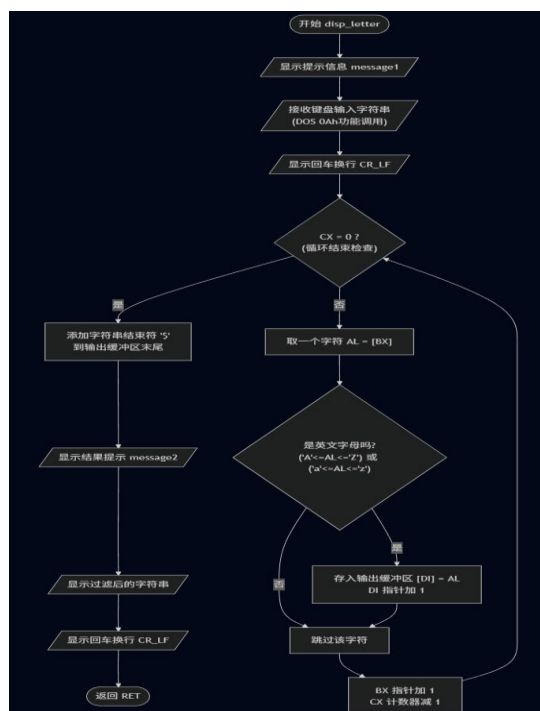
- 1) 编写源程序;
- 2) 利用 debug 命令, 使用 L、U、G 及 DDS:0 命令调试并运行程序;
- 3) 详细记录每一步所用的命令, 查看结果的方法和具体结果, 体现内存中源地址数据搬移到目标地址的效果。

3. 实验内容

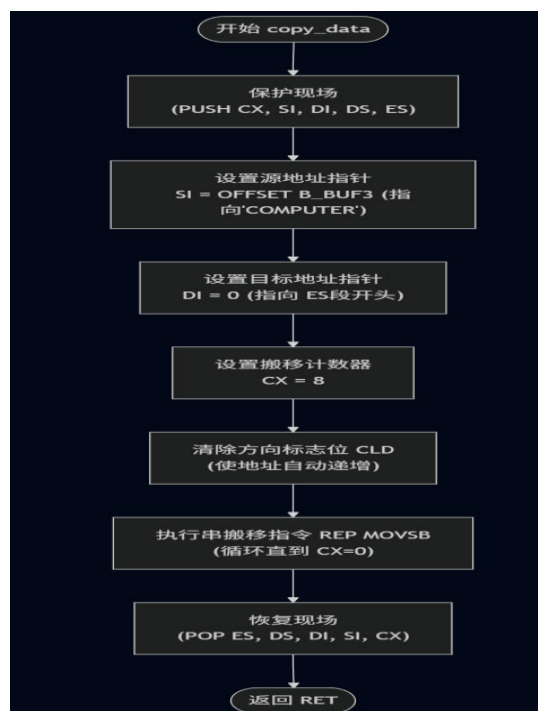
通过调试给定的 EXP1.asm, 编写并完善 copy_data 子程序。程序功能包括:

- 字符过滤: 输入字符串, 过滤并显示其中的英文字母。
- 数据搬移: 将数据段 (DATA) 中定义的变量 B_BUF3 (内容为字符串 'COMPUTER') 复制到附加段 (EDATA) 的起始位置。

4. 实验流程图



disp_letter 子程序流程图



copy_data 子程序流程图



主函数流程图

5. 实验过程与结果分析

1.代码设计:

在源代码末尾补充了 `copy_data` 子程序。该子程序使用 `SI` 寄存器作为源变址寄存器指向 `B_BUF3`，使用 `DI` 寄存器作为目的变址寄存器指向 `ES:0`。利用 `MOV CX, 8` 设置循环计数器，并通过 `REP MOVSB` 指令实现 8 个字节的连续搬移。

2.调试过程:

首先进行地址查找：加载程序后，使用 `U` 命令查看代码段。如图 1 所示，发现 `CALL disp_letter` 位于偏移地址 `0048`，而 `CALL copy_data` 位于偏移地址 `00A1`（填你实际的地址）。

```

-u
07C5:0000 1E          PUSH    DS
07C5:0001 33C0         XOR     AX,AX
07C5:0003 50          PUSH    AX
07C5:0004 B87007       MOV     AX,0770
07C5:0007 8ED8        MOV     DS,AX
07C5:0009 B89507       MOV     AX,0795
07C5:000C 8EC0        MOV     ES,AX
07C5:000E 8BD9        MOV     BX,CX
07C5:0010 8AC8        MOV     CL,AL
07C5:0012 B303        MOV     BL,03
07C5:0014 BD0330   MOV     BP,3003
07C5:0017 26          ES:
07C5:0018 C607FD       MOV     BYTE PTR [BX],FD
07C5:001B 26          ES:
07C5:001C C74701FDFF   MOV     WORD PTR [BX+01],FFFD

```

图 1-使用 `u` 指令查看反汇编代码

序输出 abcXYZ，验证了 disp_letter 子程序对非字母字符的过滤功能。

```
-g
Please input char : Hello123
result           : Hello

Program terminated normally
-
```

图 5-测试输入字符

从图中我们可以得出测试输入字符 Hello123，程序输出 Hello，验证了 disp_letter 子程序对非字母字符的过滤功能。

6. 实验心得

通过本次汇编语言编程基础实验，我不仅完成了代码编写与调试，还在以下几个方面有了深刻的理解：

1. 深入理解了 8086 系统的内存分段与寻址机制 在编写 copy_data 子程序时，我实际操作了数据段（DATA）与附加段（EDATA）之间的数据传输。这让我明白了为什么在程序初始化时必须正确设置 DS 和 ES 段寄存器。同时，通过调试主程序中的演示代码，我直观地对比了立即数寻址、寄存器寻址和直接寻址的区别，掌握了如何通过 LEA 和 OFFSET 指令获取变量的有效地址。

2. 掌握了串操作指令与循环控制的高效应用 在实现数据搬移功能时，我没有使用低效的单字节 MOV 循环，而是采用了串操作指令 REP MOVSB。实验结果证明，配合 SI（源变址）、DI（目的变址）和 CX（计数器）寄存器，可以极高效率地完成内存块的复制。这一过程加深了我对 CPU 内部寄存器分工协作的认识。此外，在分析 disp_letter 子程序时，我复习了 CMP 比较指令配合 JB/JA 等条件转移指令实现字符过滤的逻辑，理解了汇编语言中分支结构和循环结构（LOOP）的底层实现原理。

3. 熟练运用 DEBUG 工具进行动态调试 本次实验最大的收获在于掌握了动态调试方法。我学会了使用 U 命令反汇编以确定子程序的入口地址；利用

G 命令配合断点地址，让程序快速运行到关键位置；使用 T 命令单步跟踪指令执行，观察寄存器（特别是 AX, CX, SI, DI）的实时变化；以及最关键的——使用 D 命令查看 DS:OFFSET 和 ES:OFFSET 的内存内容。这种“运行前查看-运行后对比”的调试手段，是验证汇编程序正确性的唯一标准，帮助我确认了字符串 'COMPUTER' 确实被成功复制到了附加段。

4. 规范化汇编程序设计的必要性 通过阅读 EXP1.asm 的源码结构，我认识到了标准汇编程序框架的重要性，包括完整的段定义（STACK, DATA, EDATA, CODE）以及程序入口和出口的标准化写法。这为后续进行更复杂的综合程序设计（如实验二中的数据处理与文本排序）打下了坚实的基础。

实验二 8255A 可编程并行接口实验

1. 实验目的

- 1) 掌握并行接口芯片 8255A 和微机接口的连接方法；
- 2) 掌握并行接口芯片 8255A 的工作方式及其编程方法；
- 3) 掌握小键盘的工作原理及接口技术；
- 4) 利用 8255A 并行接口芯片与小键盘的接口电路，实现并行接口实验。

2. 实验原理和内容

1) 8255A 芯片原理

8255A 是可编程并行接口芯片，双列直插式封装，用+5V 单电源供电，如图 2-1 是 8255A 的逻辑框图，内部有 3 个 8 位 I/O 端口：A 口、B 口、C 口；也可以分为各有 12 位的两组：

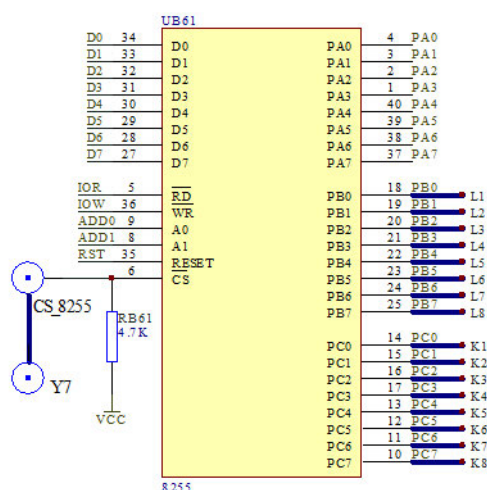


图 2-1 8255A 芯片管脚示意图

A 和 B 组，A 组包含 A 口 8 位和 C 口的高四位，B 组包含 B 口 8 位和 C 口的低 4 位。

A 组控制和 B 组控制用于实现方式选择操作；读写控制逻辑用于控制芯片内寄存器的数据和控制字经数据总线缓冲器送入各组接口寄存器中。由于 8255A 数据总线缓冲器是双向三态 8 位驱动器，因此可以直接和 8088 系统数据总线相连。

2) 8255A 端口地址

A1	A0	/RD	/WR	/CS	操作类型	操作方向
0	0	0	1	0	PA→数据总线	输入(读)
0	1	0	1	0	PB→数据总线	
1	0	0	1	0	PC→数据总线	
0	0	1	0	0	数据总线→PA	输出 (写)
0	1	1	0	0	数据总线→PB	
1	0	1	0	0	数据总线→PC	
1	1	1	0	0	数据总线→控制字	
×	×	×	×	1	数据总线三态	断开
1	1	0	1	0	非法状态	
×	×	1	1	0	数据总线三态	

图 2-2 8255A 端口地址

3) 8255A 工作方式

8255A 芯片有三种工作方式：方式 0、方式 1、方式 2。它通过对控制寄存器写入不同的控制字来决定其三种不同的工作方式。

● 方式 0：基本输入/输出

该方式下的 A 口 8 位和 B 口 8 位可以由输入的控制字决定为输入或输出，C 口分成高 4 位(PC7~PC4)和低 4 位(PC3~PC0)两组，也有控制字决定其输入或输出。需注意的是：该方式下，只能将 C 口其中一组的四位全部置为输入或输出。

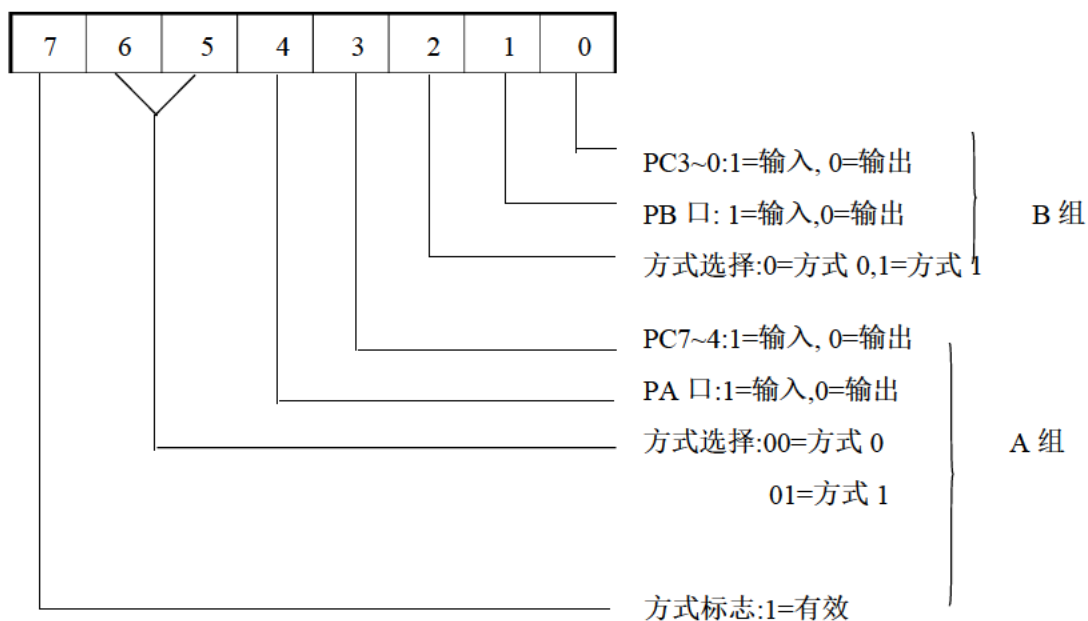
● 方式 1：选通输入/输出

该方式又叫单向输入输出方式，它分为 A、B 两组，A 组由数据口 A 和控制口 C 的高 4 位组成，B 组由数据口 B 和控制口 C 的低 4 位组成。数据口的输入/输出都是锁存的，与方式 0 不同，由控制字来决定它作输入还是输出。C 口的相应位用于寄存数据传送中所需的状态信号和控制信息。

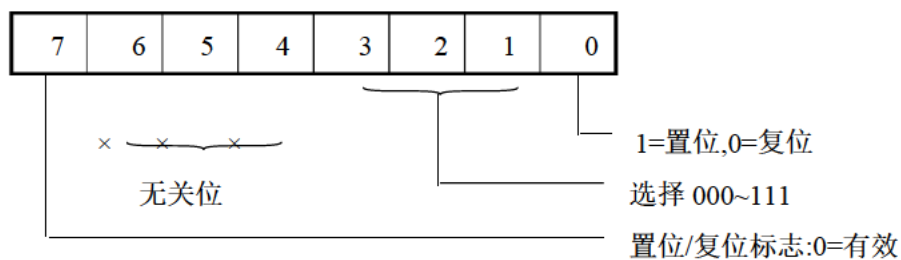
● 方式 2：双向输入输出

本方式只有 A 组可以使用，此时 A 口为输入输出双向口，C 口中的 5 位(PC3~PC7)作为 A 口的控制位。

4) 8255A 基本控制字



5) 端口 C 复位/复位控制字



6) 8255A 与小键盘连接示意图

键盘阵列由 8255 芯片控制，用行列连接方式到 8255 芯片 A 口，B 口，同时连接 8255 芯片的片选到 Y1/IO 地址（288H~28FH），以使键盘阵列由 8255 芯片控制。如图 2-3 所示：

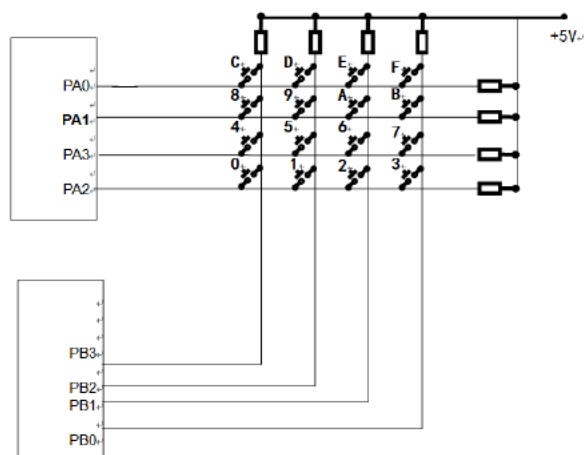


图 2-3 8255A 与小键盘连接示意图

7) 实验硬件连接方法

键盘矩阵由 8255A 芯片控制，并将 PA0-PA3、PB0-PB3 与键盘连接到 8255A 的 A 口和 C 口，同时片选地址 288H-28FH，以便键盘矩阵由 8255A 控制。

- 8255A 的 PA0-PA3 连接小键盘的行 0-行 3；
- 8255A 的 PB0-PB7 连接小键盘的列 0-行 3；
- 8255A 的 CS 连接实验箱 Y1/IO 地址。从 288H-28FH 选择。

3. 实验程序

编写程序,按下小键盘任意键后,要求在用 2 号调用在计算机屏幕显示结果。

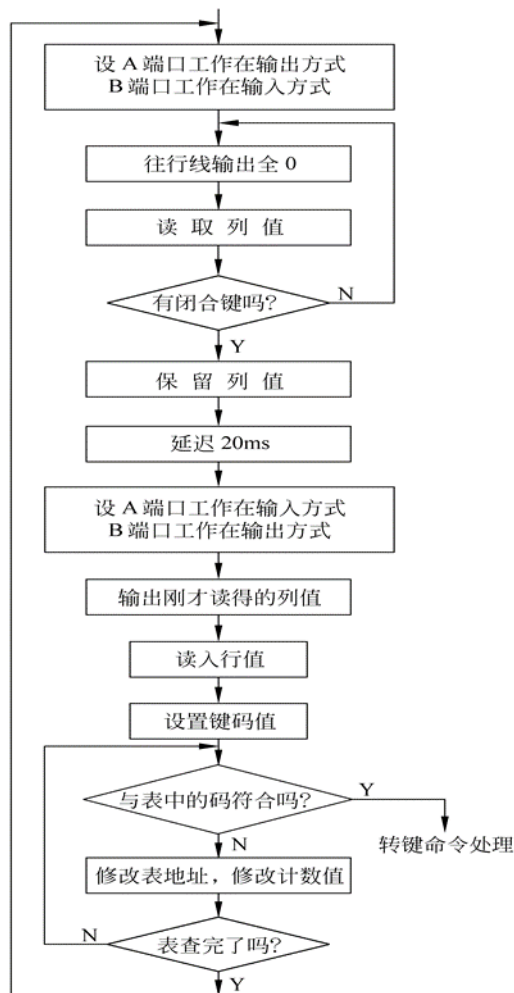


图 3-1 小键盘显示设计流程图

具体程序如下：

```
1. data    segment
2.         tab1    DB  'F', 'E', 'D', 'C'
```

```

3.      tab2    DB  '3', '2', '1', '0'
4.      tab3    DB  'B', 'A', '9', '8'
5.      tab4    DB  '7', '6', '5', '4'
6.      kb      DB  00eh, 00dh,00bh,007h
7.      port    equ  288h
8. data      ends
9.
10. stack    segment
11.      sta     db  256 dup(?)
12.      top     equ  size sta
13. stack    ends
14. code     segment
15.      assume cs:code,ds:data,ss:stack
16. main proc far
17.      mov ax,data
18.      mov ds,ax
19.      mov ax, stack
20.      mov ss,ax
21.      mov sp,top      ; 初始化 8255
22.      mov dx, port+3
23.      mov al,10000010b
24.      out dx,al
25. begin:
26.      mov al,0
27.      mov dx, port
28.      out dx,al
29.      mov dx, port+1
30.      in al,dx
31.      and al,0fh
32.      cmp al,0fh

```

33. je begin ;若有键按下，按列查找哪个键

34.

35. mov al,00001110b

36. mov dx, port

37. out dx,al

38. mov dx, port+1

39. in al,dx

40. lea bx,tab1

41. cmp al,0fh

42. jne display

43. mov al,00001101b

44. mov dx, port

45. out dx,al

46. mov dx, port+1

47. in al,dx

48. lea bx,tab2

49. cmp al,0fh

50. jne display

51. mov al,00001011b

52. mov dx, port

53. out dx,al

54. mov dx, port+1

55. in al,dx

56. lea bx,tab3

57. cmp al,0fh

58. jne display

59. mov al,00000111b

60. mov dx, port

61. out dx,al

62. mov dx, port+1

```

63.    in al,dx
64.    lea bx,tab4
65.    cmp al,0fh
66.    jne display
67.    nop
68.    jmp begin      ;查找按下的键，并显示
69. display:
70.    mov di,bx
71.    lea si,kb      ;存小键盘扫描码
72.
73. next:
74.    cmp al,[si]    ;查键盘扫描码表
75.    je show        ;找到了,转去显示
76.    inc si          ;否则继续
77.    inc di
78.    jmp next
79.
80. show:              ;显示键入字符
81.    mov ah,2
82.    mov dl,[di]
83.    int 21h
84.    cmp dl, 'F'     ;若是'P'则退出
85.    je over
86.    call delay
87.    jmp begin      ;重新开始查询
88.
89.
90. over:
91.    mov ah,4ch
92.    int 21h

```

```
93. main endp
94.
95. delay proc near ;延时循环 100*65536 次
96.     push di
97.     push cx
98.     mov cx,0fh
99.     mov di,0
100.
101. again:
102.     nop
103.     dec di
104.     jnz again
105.     dec cx
106.     jnz again
107.
108.     pop cx
109.     pop di
110.     ret
111. delay endp
112.
113. code ends
114. end main
```

4. 实验结果

按照讲义中实验硬件连接方法将硬件连接好，启动 TPC-ZK-II 通用微机接口实验系统，指示灯全部正常亮起，如图 4-1 所示：



图 4-1 工作中的 TPC-ZK-II 通用微机接口实验系统

打开 TPC-ZK-II 集成开发环境，选择“选项”中的“硬件检测”，提示硬件已连接，证明设备无误，可以开始进行实验。

- 1) 在当前运行环境下，选择菜单栏中的“文件”菜单，菜单下拉后选择“新建”，出现源程序编辑窗口，将代码写入编辑窗口中。
- 2) 再次选择菜单栏中的“文件”菜单，菜单下拉后选择“保存”，为文件取名保存后，成功新建一个“.asm”文件。
- 3) 选择菜单栏中的“项目”菜单，选择汇编+链接选项，则环境对当前 ASM 源文件进行汇编与链接，编译调试窗口中输出汇编与链接的结果，显示成功编译与链接。
- 4) 再次选择菜单栏中的“项目”菜单，选择开始+执行选项，则环境对当前 ASM 源文件执行，程序自动运行。

至此，程序成功运行并弹出运行窗口，等待输入。实验三与实验四的软件操作部分与此部分相同，故后面不再赘述此部分的操作。

从 TPC-ZK-II 通用微机接口实验系统左下角的小键盘输入不同的符号，观察运行窗口的输出。

首先，输入符号“9”，如图 4-2 所示：

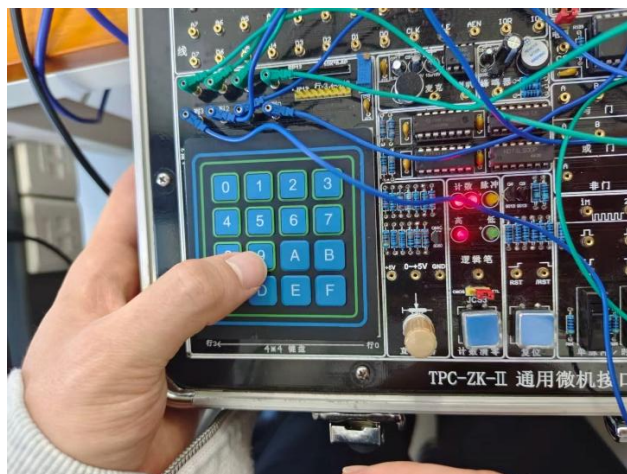


图 4-2 从小键盘上输入符号“9”

观察运行窗口的变化，结果为屏幕上输出了“9999”，如图 4-3 所示：

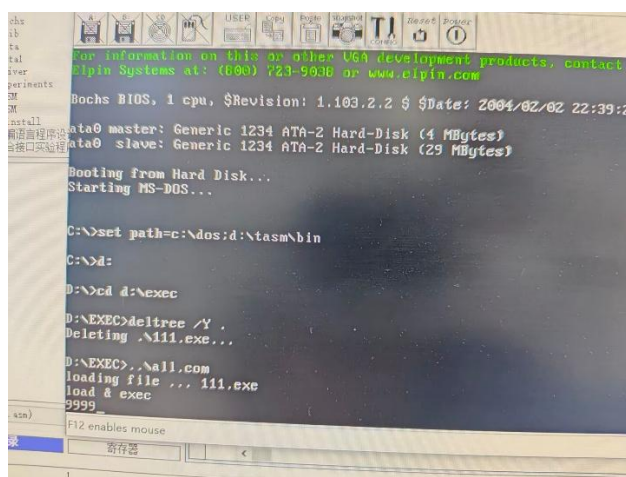


图 4-3 运行窗口输出

可以看到，输入符号“9”，期待的输出为符号“9”，故结果是正确的，显示 4 个符号“9”的原因是没有做按键消抖部分，系统判定为连续输入了 4 个符号“9”，故输出“9999”，但不影响结果的正确性。

为了验证所有字符都能对应输出，接着输入符号“E”，如图 4-4 所示：

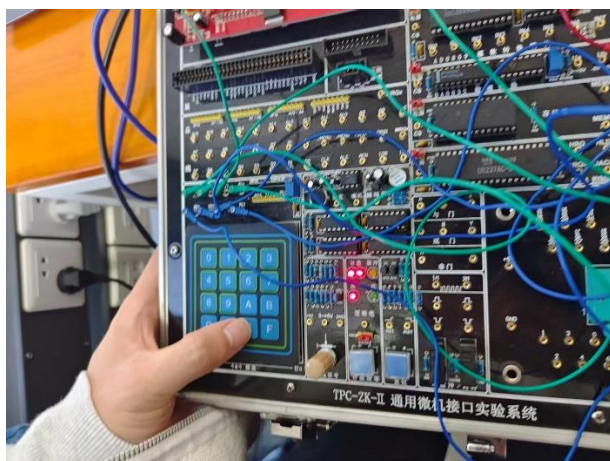


图 4-4 从小键盘上输入符号“E”

观察运行窗口的变化，结果为屏幕上输出了“EEEE”，如图 4-5 所示：

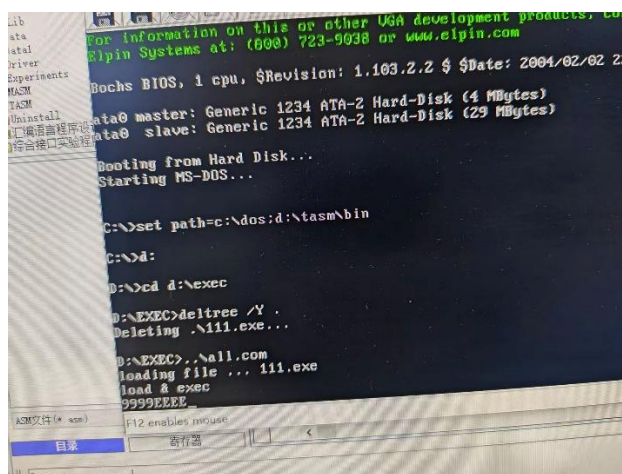


图 4-5 运行窗口输出

可以看到，输入符号“E”，期待的输出为符号“E”，故结果是正确的。
然后，输入符号“0”，如图 4-6 所示：



图 4-6 从小键盘上输入符号“0”

观察运行窗口的变化，结果为屏幕上输出了“0000”，如图 4-7 所示：

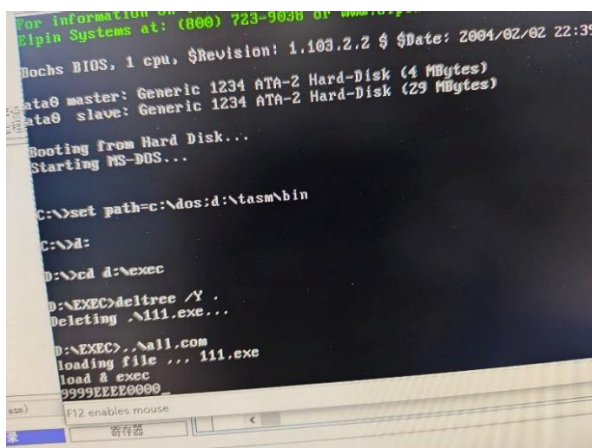


图 4-7 运行窗口输出

可以看到，输入符号“0”，期待的输出为符号“0”，故结果是正确的。

至此，我们测试了三个不同行、不同列的符号，均输出正确，证明硬件连线和软件程序对小键盘操控是正确的。

最后，输入符号“F”，如图 4-8 所示：

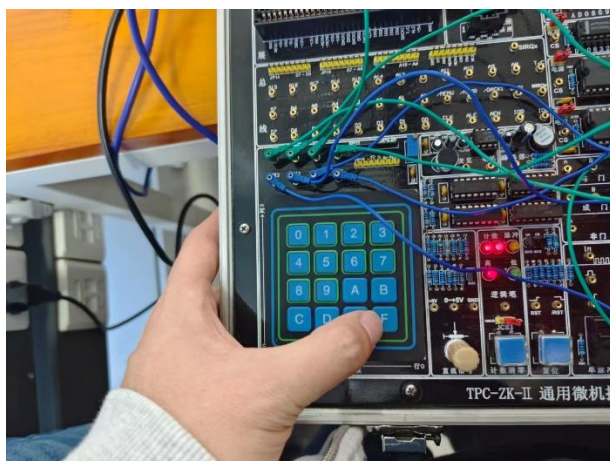


图 4-8 从小键盘上输入符号“F”

观察运行窗口的变化，结果为屏幕上输出了“F done!”，如图 4-9 所示：

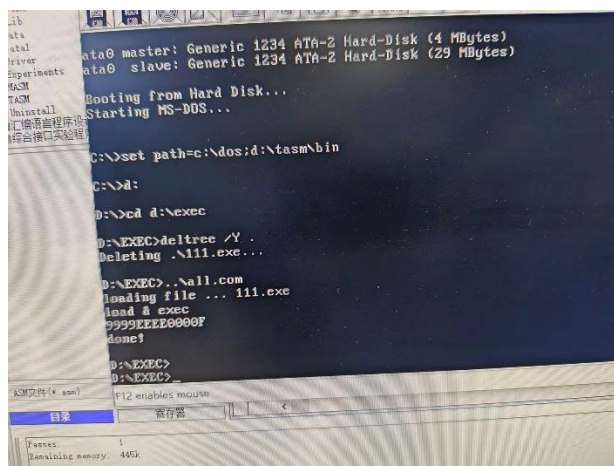


图 4-9 运行窗口输出

根据实验讲义，输入符号“F”，期待的输出为符号“F”并结束输入，故结果是正确的。并且此次只输出一个符号“F”，进一步证明了此前连续四个符号输出的原因是无按键消抖。

至此，本小组实验二的输出结果完全正确，证明此实验的硬件连接、软件程序是正确的。

5. 实验分析 ()

本次《8255A 可编程并行接口实验》不仅让我成功实现了小键盘输入与屏幕显示的基本功能，更使我对并行接口的工作机制、硬件与软件的协同设计有了系统而深入的理解。以下是我对本次实验的全面分析与总结：

1) 对 8255A 芯片工作方式的理解与应用

实验的核心在于正确配置 8255A 的工作方式。我们通过向控制口（端口地址 28BH）写入控制字 10000010B 将其初始化为方式 0。这个控制字的每一位都有明确含义：

- D7=1：标志位，表明这是一个设置工作方式的控制字
- D6D5=00：设定 A 组（A 口和 C 口高四位）工作在方式 0
- D4=0：设定 A 口为输出模式，用于向键盘矩阵的行线发送扫描信号
- D3=0：设定 C 口高四位为输出，在本实验中未使用
- D2D1=10：设定 B 组（B 口和 C 口低四位）工作在方式 0
- D0=0：设定 B 口为输入模式，用于读取键盘矩阵的列线状态

这种 A 口输出、B 口输入的配置，完美契合了键盘矩阵扫描的行扫列读基本需求，是实验成功的关键第一步。

2) 键盘扫描逻辑的深入剖析与实现

程序的核心逻辑是键盘扫描。我理解其本质是一个主动查询的过程：

- 扫描信号输出：程序通过 A 口循环输出四种不同的扫描字（00001110B,00001101B,00001011B,00000111B），这相当于依次将第 0 行、第 1 行、第 2 行、第 3 行拉低（置 0），而其他行保持高电平（置 1）。
- 状态信号读取：每次输出一行扫描信号后，立即从 B 口读取列值。由于硬件上接了上拉电阻，无按键时读入的列值应为 0FH。如果该行有按键被按下，则对应的列线会因为与低电平的行线连通而被拉低，读入的列值就不再是 0FH。
- 按键定位：一旦发现列值非 0FH，就结合当前扫描的行号和读到的列号，唯一确定一个按键。程序中通过 LEA BX, tabX 将不同行对应的字符表首地址装入 BX，再通过查扫描码表 kb 进行列匹配，最终在字符表中定位到正确的按键字符。这个过程清晰地体现了从硬件信号到软件数据的转换。

3) 硬件调试与现象分析能力

在实验结果中，我们观察到了一个关键现象：按下一次按键，屏幕会输出多个相同字符。我深入分析了这一现象的根本原因：按键抖动。

- 原因：机械按键在闭合和断开的瞬间会产生一系列不稳定的抖动信号，虽然人眼无法察觉，但 CPU 的扫描速度极快，会在一次按压期间多次检测到有效的按下状态。
- 验证：这个现象并非程序逻辑错误，恰恰证明了我们的扫描程序是高效且连续工作的。最后的“F”键只显示一次，是因为程序在显示“F”后直接退出，没有机会进行下一次扫描，从而避免了抖动的影响。
- 解决方案：在真实的嵌入式系统中，必须加入软件消抖或硬件消抖。软件消抖可以在检测到按键后，插入一个 10-20ms 的延时，再重新检测按键是否仍处于稳定按下状态，从而过滤掉抖动期的信号。

4) 综合实践能力的提升

通过本次实验，我将理论课上学到的并行接口知识付诸实践，完成了从芯片选型、接口设计、程序编写到系统调试的完整流程。我不仅掌握了 8255A 的编程方法，更重要的是建立了软硬件不分家的系统级设计思想。例如，程序中 `tab1` 到 `tab4` 字符表的定义，必须与硬件上键盘的物理布局严格对应，任何一者的错误都会导致按键与显示字符错乱，这锻炼了我严谨的工程思维。

5) 总结

本实验是一次非常成功的综合设计实践。我不仅实现了所有基础功能，还通过观察和分析异常现象（按键抖动），挖掘出了更深层次的工程问题并提出了解决方案。这充分证明了我对 8255A 可编程并行接口的工作原理、键盘扫描算法以及汇编语言编程有了扎实的掌握，为后续学习更复杂的接口技术奠定了坚实的基础。

6. 实验分析 ()

在 8255A 可编程并行接口实验中，我们小组以小键盘输入与屏幕显示为实现的功能目标，完成了从汇编语言代码编写调试到硬件接口搭建的完整实验流程。该实验不仅验证了 8255A 并行接口芯片的工作原理机制，更让我们对硬件电路特性与软件代码编写之间的协同关系有了更深刻的理解，以下从技术角度对本次实验展开详细的分析与总结：

1) 8255A 工作方式的配置逻辑

8255A 的工作模式直接决定了其与外部设备的交互方式，本实验能够顺利实现功能，首先要对 8255A 进行正确的初始化设置。通过向控制端口写入控制字 `10000010B`，将 8255A 初始化为方式 0。其中控制端口地址为 `28BH`，对应 8255A 控制寄存器。该控制字的每一位都对应明确的硬件功能定义，高度契合实验需求：

- `D7=1`：控制字标志位，表明该指令为工作方式设置指令，而非端口 C 的置位/复位指令。
- `D6D5=00`：定义 A 组工作在方式 0，表明无需握手信号。
- `D4=0`：实验中 A 口 `PA0~PA3` 直接连接小键盘的第 0 行~第 3 行，用于循环输出行扫描信号，拉低目标行以检测列状态。

-
- D3=0: 设定 C 口高 4 位为输出模式, 本实验未使用。
 - D2D1=10: 定义 B 组工作在方式 0, 表明无需握手信号。
 - D0=0: 设定 B 口为输入模式, 用于实时读取列线电平状态, 判断是否有按键按下。

这种 A 口输出负责行扫描、B 口输入负责列采集的配置, 与上述小键盘的检测原理完全相符, 为后续键盘扫描逻辑的实现做好了充分的准备。

2) 键盘扫描的实现机制

上述汇编语言程序的核心逻辑是键盘扫描, 主要通过查询的方式来实现, 具体过程可分为 3 个步骤:

- 行扫描信号的循环输出。为了逐一检测每一行的按键状态, 程序通过 A 口向行线输出 4 组特定扫描字。其中 00001110B 对应拉低第 0 行其余行保持高电平; 00001101B 对应拉低第 1 行其余行保持高电平; 00001011B 对应拉低第 2 行 其余行保持高电平; 00000111B 对应拉低第 3 行其余行保持高电平。该设计的核心逻辑是逐行激活, 每次仅让一行处于低电平。该行存在按键按下, 对应的列线会因与行线导通而被拉低, 进而通过列状态读取定位按键。
- 列状态信号的实时采集。在硬件电路中, 小键盘列线均接有上拉电阻, 无按键按下时, 列线保持高电平, 此时从 B 口读取的列值为 0FH 也就是二进制 11111111; 若某行存在按键按下, 导通的列线会被拉低, 读取的列值将出现 0 位。程序在输出每组扫描字后, 立即通过 IN AL DX 指令读取列值, 其中 DX 指向 B 口地址, 随后通过 AND AL 0FH 指令屏蔽高 4 位干扰, 确保列状态检测的准确性。
- 按键字符的匹配定位。当检测到列值非 0FH 时, 程序通过行号与列值双重信息定位按键字符。首先根据当前输出的扫描字确定行号, 接着通过 LEA BX tabX 指令将对应行的字符表首地址装入 BX, 随后通过扫描码表 kb 存储列值与表索引的对应关系匹配列值, 最终从字符表中提取对应的按键字符, 完成硬件信号到软件字符的转换。

3) 实验现象分析与问题解决思路

在实验调试过程中, 我们观察到一个现象: 单次按下小键盘按键, 屏幕会连

续输出多个相同字符,例如按下 9 会输出 9999,仅按下 F 键时仅输出 1 次 F。我们对这一现象进行了深入分析,发现是由按键抖动引起的。

机械按键的触点在闭合与断开瞬间,会因弹性形变产生 10 至 20 毫秒的不稳定抖动信号。该信号持续时间远短于人眼感知范围,但 CPU 执行速度极快,本实验中扫描周期远小于抖动时间,会在抖动期间多次检测到按键按下的有效状态,进而导致连续输出字符。

但是在程序设计中,F 键为退出触发键,程序在检测到 F 后立即执行 JE OVER 指令退出扫描循环,无后续扫描操作,因此仅能捕获一次有效按键信号,仅输出 1 次 F。该结果反证了字符连续输出是扫描程序持续工作导致,并非软件逻辑错误。

在工程实践中,需要进行消抖处理以过滤抖动信号,本实验场景下采用软件消抖方案。在首次检测到列值非 0FH 疑似按键按下后,插入 10 至 20 毫秒的软件延时,该延时可通过 DELAY 子程序实现,待抖动结束后再次读取列值;若两次读取的列值一致,确认按键稳定按下,再执行字符匹配与显示,避免抖动信号导致的误判。

4) 实验总结

本次 8255A 可编程并行接口实验,对我而言是一次极具价值的综合实践历练。在从方案构思到最终落地调试的完整实验过程中,我真正实现了将课堂中学到的 8255A 工作原理与实际操作相结合,不再是单纯记忆知识点,而是在实践的基础上深化对知识的理解。本次实验中我不仅熟练掌握了 8255A 接口芯片的使用方法,更体会到了软硬件之间密不可分的关联,这让我跳出了单一的知识维度,建立起系统化的思考方式。在实践中我深刻体会到,任何环节的疏漏都可能影响最终实验效果,这就要求我在每一步操作中都要认真细致,尽可能避免错误的发生。面对实验中遇到的问题时,我没有停留在问题的表面,而是主动深入分析问题出现的根源并探寻解决办法,这个过程不仅锻炼了我的问题排查与解决能力,更让我体会到主动探究的意义。此次实验不仅夯实了我对 8255A 可编程并行接口的工作原理、键盘扫描的实现方法以及汇编语言编程等相关知识的理解与掌握,更让我积累了宝贵的综合实践经验,为后续接触并学习更复杂的理论和技术奠定了坚实基础,也让我对工程实践有了更深刻的认识与感悟。

7. 实验分析 ()

本次 8255A 可编程并行接口实验让我从硬件连接到软件编程都经过了锻炼，掌握了并行接口与外部设备交互的核心逻辑。通过实现小键盘输入与屏幕显示功能，我对 8255A 芯片的工作机制和键盘扫描原理有了扎实的实践认知。

此次实验成功的关键在于精准配置 8255A 的工作方式。我们将控制字 10000010B 写入控制端口，而 8 位控制字的每一位都对应着端口的功能定义：D7 位为 1 表示是方式控制字，A 口+PC 高 4 位设为方式 0 输出，B 口+PC 低 4 位设为方式 0 输入。这种配置匹配键盘矩阵行扫列读的需求——A 口作为输出端口向行线发送扫描信号，B 口作为输入端口读取列线状态，两者协同实现按键检测。在调试过程中，我曾因误写控制字导致端口功能异常，通过反复核对控制字每位的含义，最终确认了正确配置，这让我深刻体会到软件定义硬件的含义。

键盘扫描是实验的核心流程之一。程序先通过 A 口输出全 0 信号，若 B 口读取到非 0FH 的列值，说明有按键按下；随后通过循环输出不同的行扫描字，逐行拉低行线，结合读取的列值锁定按键位置。在这个过程中，字符表 tab1-tab4 的定义必须与键盘物理布局严格对应，否则会出现“按此键显彼字”的错误。通过调整字符表顺序与键盘实际按键位置匹配，最终实现了输入与显示的一致性，这让我认识到软硬件逻辑对齐的重要性。

通过本次实验，我不仅掌握了 8255A 芯片的编程方法和键盘矩阵的扫描原理，更提升了软硬件协同设计的思维能力。从硬件连接时 PA0-PA3 与行线、PB0-PB7 与列线的对应，到软件中控制字配置、扫描逻辑编写、字符表定义，每个环节都需要严谨的逻辑推导。同时，我也深刻认识到理论知识与实践结合的重要性，只有将理论内容应用到编程中，才能真正理解其工作机制。

8. 实验分析 ()

1) 硬件接口与工作方式配置的深入理解

本次实验的核心在于利用 8255A 芯片实现对矩阵键盘的控制。在初始化阶段，我们向控制口（28BH）写入了控制字 10000010B。这一操作不仅是简单的赋值，而是对硬件行为的精确定义：

- D7=1：标志着这是方式设置命令，而非位复位命令。

-
- A 口 (D4=0): 配置为输出模式。在物理连接上, PA0-PA3 连接了键盘的行线。这决定了我们的扫描策略必须是“主动输出行信号”。
 - B 口 (D1=1): 配置为输入模式。PB0-PB3 连接了列线, 用于读取按键闭合后的电平反馈。

这种配置确立了“行扫描法”的基础, 即通过 A 口轮询输出低电平, 通过 B 口检测低电平来实现按键定位。

2) 软件扫描算法的逻辑分析

实验程序采用了一种高效的查表法来实现键值转换。程序逻辑并没有使用繁琐的 IF-ELSE 判断, 而是巧妙地利用了四个数据表 tab1 到 tab4 2。

- 扫描过程: 程序依次向 A 口写入 0FEH (1110B), 0FDH (1101B) 等扫描码, 依次拉低第 0 行到第 3 行。
- 定位原理: 当某一行的扫描码输出后, 程序立即读取 B 口状态。如果读到的不是 0FH (即存在低电平), 说明该行有键按下。
- 键值映射: 通过 LEA BX, tabX 指令, 我们将当前扫描行对应的字符表首地址加载到基址寄存器。这种“硬件位置”到“内存地址”的映射, 极大地简化了代码结构。

3) 实验现象与工程改进思考

在实验结果中, 我观察到一个非常典型的现象: 按下一次数字键 (如‘9’), 屏幕上输出了串“9999” 3。

- 原因分析: 这是由于机械按键的“抖动”以及 CPU 极高的执行速度造成的。CPU 扫描键盘的一个循环可能仅需几微秒, 而人手按键的动作通常持续几十毫秒。在这段时间内, 程序检测到了多次“按键闭合”状态, 并执行了多次显示调用。
- 对比验证: 当按下‘F’键时, 屏幕只显示了一次“F”并退出 4。这是因为程序设计中检测到‘F’后直接跳转到退出指令, 切断了后续的扫描循环, 从而物理上屏蔽了后续的抖动检测。
- 改进方案: 在实际工程中, 必须加入“去抖动”环节。可以在检测到按键按下后, 调用一个 10ms-20ms 的延时子程序, 然后再次检测。如果此时键仍处于闭合状态, 才确认为有效输入。

实验三 使用 8259 中断实现定时/计数器实验

1. 实验目的

- 1) 掌握定时/计数器 8254 的基本工作原理和编程方法；
- 2) 利用 TPC-ZK-II 实验板上的 8254 向中断源发出中断申请，其中断请求用 IRQ3（系统总线区的 IRQ）；
- 3) 了解七段数码管显示原理，掌握其显示接口技术，了解数字时钟原理，掌握实现数字时钟技术；
- 4) 掌握中断控制器 8259A 的应用编程。

2. 实验原理和内容

本实验的硬件中断只有可屏蔽中断，由 8259 中断控制器管理。

1) 8259 内部结构

8259A 是专为控制优先级中断而设计的芯片。它将中断源按优先级排队、辨认中断源、提供中断向量的电路集成于一体，只要用软件对它进行编程，就可以管理 8 级中断。

它由中断请求寄存器(IRR)、优先级分析器、中断服务寄存器(ISR)、中断屏蔽寄存器(IMR)、数据总线缓冲器、读写控制电路和级联缓冲器、比较器组成。

- 中断请求寄存器：寄存所有要求服务的请求 IR0~IR7。
- 中断服务寄存器：寄存正在被服务的中断请求。
- 中断屏蔽寄存器：存放被屏蔽的中断请求，该寄存器的每一位表示一个中断号，该位为 1，屏蔽该号中断，否则开放该号中断。
- 数据总线缓冲器：是双向三态的，用以连接系统总线和 8259A 内部总线，通过它可以由 CPU 对 8259A 写入状态字和控制字。
- 读写控制电路：用来接受 I/O 命令，对初始化命令和操作命令寄存器进行写入，以确定 8259A 的工作方式和控制方式。
- 级联缓冲器/比较器：用于多片 8259A 的连接，能构成多达 64 级的矢量中断系统。

2) 中断源与类型

表 2-1 中断类型号

中断源	中断类型号	中断功能
IRQ0	08H	时钟
IRQ1	09H	键盘
IRQ2	0AH	保留
IRQ3	0BH	串行口 2
IRQ4	0CH	串行口 1
IRQ5	0DH	硬盘
IRQ6	0EH	软盘
IRQ7	0FH	并行打印机

8 个中断源的中断请求信号线 IRQ0~IRQ7 在主机 62 线 ISA 总线插座中可以引出，系统已设定中断请求信号为“边沿触发”，普通结束方式。对于微机内又扩展了一片 8259 中断控制，IRQ2 用于两片 8259 之间级连，对外可以提供 16 个中断源，如表 2-2 所示：

表 2-2 扩展中断类型号

中断源	中断类型号	中断功能
IRQ8	070H	实时时钟
IRQ9	071H	用户中断
IRQ10	072H	保留
IRQ11	073H	保留
IRQ12	074H	保留
IRQ13	075H	协处理器
IRQ14	076H	硬盘
IRQ15	077H	保留

TPC-ZK-II 实验板上，固定的接到了 3 号中断 IRQ3 上，即进行中断实验时，所用中断类型号为 0BH。

中断控制器 8259 可利用系统初始化设定，如不进行新的初始化，只需对 8259 中断控制器的屏蔽寄存器设定 IRQ3 为中断允许，其他原有中断允许保持不变。

3) 8254 设置

将实验板上的 8254 的通道，设定为 10ms 周期信号发生器，该芯片的通道 1 设定为计数器，其输入来自定时器通道 0 的输出，利用通道 1 的计数器输出作为 IRQ3 中断请求信号。

4) 8254 连接图

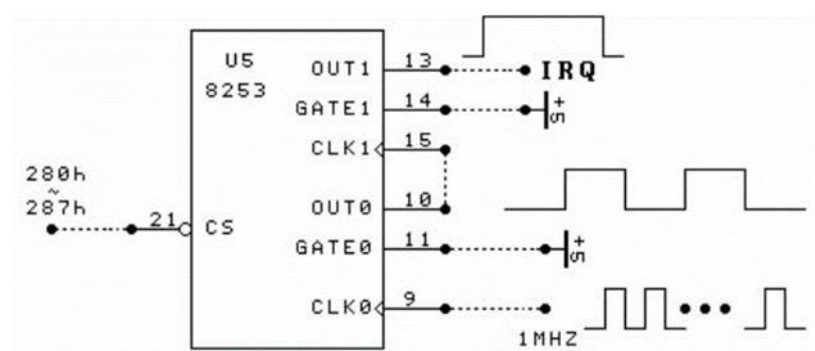


图 2-1 8254 定时器连线

8254 定时器的 CLK0 由实验板上的 1MHZ 振荡器来提供。8254 定时器的 CLK0 由实验板上的 1MHZ 振荡器来提供。

5) 数码管连接图与段码表

数码管显示接口电路如图 2-2 所示。实验台上的七段数码管为共阴型，段码采用同相驱动，输入端加高电平,选中的数码管亮，位码加反相驱动器，位码输入端高电平选中。

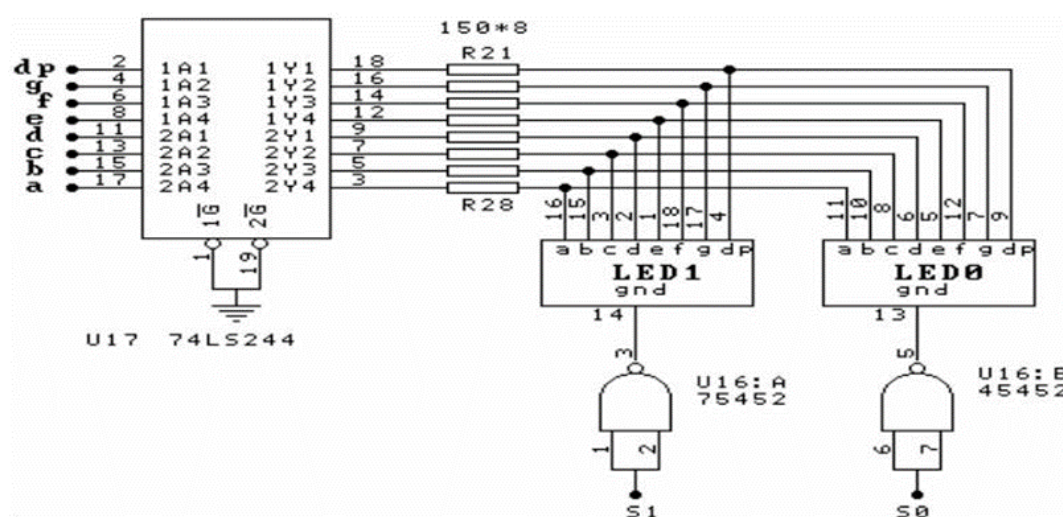
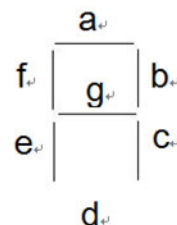


图 2-2 数码管显示电路

本实验需要 PB7~PB0/8255 接 dp~a/LED 数码管,PC3~PC0/8255 接 S3~S0/LED 数码管。七段数码管的字型代码表如表 2-3 所示：

表 2-3 七段数码管的字型代码表

显示字形	g	e	f	d	c	b	a	段码
0	0	1	1	1	1	1	1	3fh
1	0	0	0	0	1	1	0	06h
2	1	0	1	1	0	1	1	5bh
3	1	0	0	1	1	1	1	4fh
4	1	1	0	0	1	1	0	66h
5	1	1	0	1	1	0	1	6dh
6	1	1	1	1	1	0	1	7dh
7	0	0	0	0	1	1	1	07h
8	1	1	1	1	1	1	1	7fh
9	1	1	0	1	1	1	1	6fh



6) 实验硬件连接方法

- 8255 芯片的 PB7~PB0 连接 LED 数码管的 dp~a;
- 8254 芯片的片选 CS 连接 IO 地址的 Y0;
- 8255 芯片的 PC3~PC0 连接 LED 数码管的 S3~S0;
- 8255 芯片的片选 CS 连接 IO 地址的 Y1;
- 8254 芯片的 OUT1 连接 IRQ/总线;
- 8254 芯片的 GATE0 连接+5V;
- 8254 芯片的 CLK0 连接 1M 时钟;
- 8254 芯片的 OUT0 连接 8254 芯片的 CLK1;
- 8254 芯片的 GATE1 连接+5V。

3. 实验程序

编写中断服务程序，进行数字钟显示：XX: XX（分秒），其中 X 为一数码管。

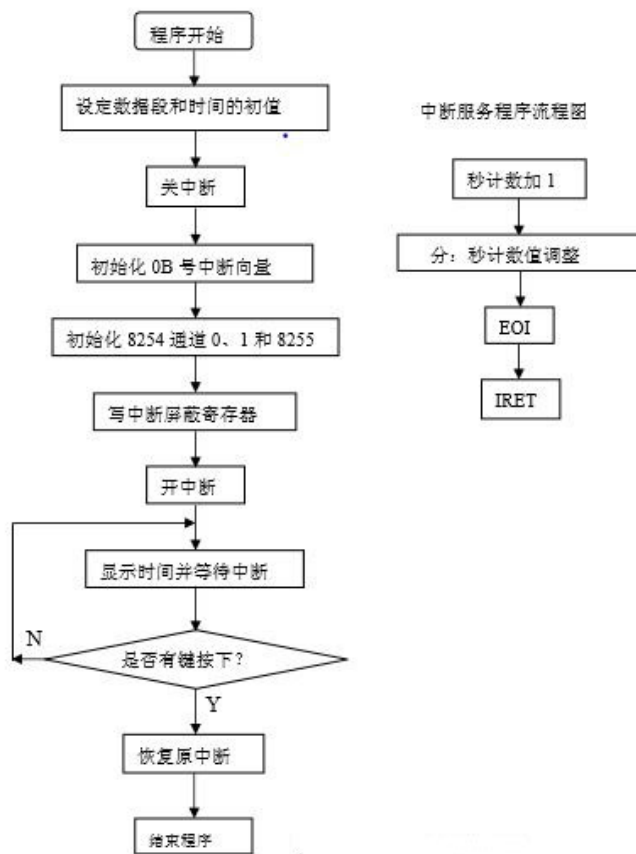


图 3-1 实验程序设计流程图

具体程序如下：

```

1. p8259 equ 20h ;端口首地址
2. p8255 equ 288h ;端口首地址
3. p8253 equ 280h ;IO Y1
4. data segment
5. led db 3fh,06h,5bh,4fh,66h,6dh,7dh,07h,7fh,6fh ;数码管
6. sec1 db 5
7. sec2 db 5
8. min1 db 9
9. min2 db 5
10. hour1 db 3
11. hour2 db 2
12. mes db 42h

```



```

13.      Old_0A      DW  0,0    ;原中断向量
14.      Old_8259     DB  0
15. data ends
16. stacks segment
17.      db 64 dup (0)
18. stacks ends
19. code segment
20.      assume cs:code,ds:data,es:data
21. start:
22.      mov ax,data
23.      mov ds,ax
24.      cli;关闭中断
25.      mov al,00110110b
26.      mov dx,p8253+3    ;计数器 0,工作在模式 3
27.      out dx,al
28.      mov ax,10000
29.      mov dx,p8253
30.      out dx,al
31.      mov al,ah          ;先赋高八位 再赋低八位
32.      out dx,al
33.
34.      mov al,01110110b
35.      mov dx,p8253+3    ;计数器 1,工作在模式 3
36.      out dx,al
37.      mov ax,100
38.      mov dx,p8253+1
39.      out dx,al
40.      mov al,ah          ;先赋高八位 再赋低八位
41.      out dx,al          ;设置中断向量表
42.

```

```

43.    PUSH ES
44.    PUSH BX
45.    MOV AL,0BH    ;保护原中断向量,0B 中断类型号
46.    mov ah,35h
47.    INT 21H        ;读中断向量表
48.    MOV Old_0A+2,ES    ;ES 为段地址,BX 为段偏移地址
49.    MOV Old_0A,BX
50.
51.    POP BX
52.    POP ES
53.    push ds
54.    mov al,0bh    ;  写新中断向量
55.    mov ah,25h
56.    mov dx,seg INT_0A    ;读取段地址保存
57.    mov ds,dx
58.    mov dx,offset INT_0A    ;读取偏移量保存
59.    int 21h
60.
61.    pop ds
62.    mov al,10000000b    ;对 8255 进行设置,PB/PC 输出
63.    mov dx,p8255+3
64.    out dx,al
65.    in al,p8259+1    ;8259 奇地址中断向量屏蔽
66.    mov old_8259,al    ;字设置    保护原屏蔽字
67.    and al,11110111b    ;开 IRQ3
68.    out p8259,al
69.    sti                ;开中断
70.
71. play:
72.    lea bx,led        ;led 段码

```

```

73.    mov dl,sec1      ;秒个位
74.    mov dh,0
75.    mov si,dx
76.    mov al,[bx][si]
77.    mov dx,p8255+1   ;点亮 led
78.    out dx,al
79.    mov ah,11111110b
80.    mov al,ah
81.    mov dx,p8255+2   ;pc 输出选通信号
82.    out dx,al
83.    PUSH CX
84.    MOV CX,00f00h ;延时
85.
86. delay1:
87.    LOOP delay1
88.    POP CX
89.    mov ah,11111111b
90.    mov al,ah
91.    mov dx,p8255+2   ;pc 输出选通信号
92.    out dx,al
93.    mov dl,sec2      ;秒十位
94.    mov dh,0
95.    mov si,dx
96.    mov al,[bx][si]
97.    mov dx,p8255+1   ;点亮 led
98.    out dx,al
99.    mov ah,11111101b
100.   mov al,ah
101.   mov dx,p8255+2   ;pc 输出选通信号
102.   out dx,al

```

```

103.    PUSH CX
104.    MOV CX,00f00h ;延时
105.
106. delay2:
107.    LOOP delay2
108.    POP CX
109.    mov ah,11111111b
110.    mov al,ah
111.    mov dx,p8255+2 ;pc 输出选通信号
112.    out dx,al
113.    mov dl,min1 ;分个位
114.    mov dh,0
115.    mov si,dx
116.    mov al,[bx][si]
117.    mov dx,p8255+1 ;点亮 led
118.    out dx,al
119.    mov ah,11111011b
120.    mov al,ah
121.    mov dx,p8255+2 ;pc 输出选通信号
122.    out dx,al
123.    PUSH CX
124.    MOV CX,00f00h ;延时
125.
126. delay3:
127.    LOOP delay3
128.    POP CX
129.    mov ah,11111111b
130.    mov al,ah
131.    mov dx,p8255+2 ;pc 输出选通信号
132.    out dx,al

```

```

133.    mov dl,min2      ;分十位
134.    mov dh,0
135.    mov si,dx
136.    mov al,[bx][si]
137.    mov dx,p8255+1   ;点亮 led
138.    out dx,al
139.    mov ah,11110111b
140.    mov al,ah
141.    mov dx,p8255+2   ;pc 输出选通信号
142.    out dx,al
143.    PUSH CX
144.    MOV CX,00f00h ;延时
145.
146. delay4:
147.    LOOP delay4
148.    POP CX
149.    mov ah,11111111b
150.    mov al,ah
151.    mov dx,p8255+2   ;pc 输出选通信号
152.    out dx,al
153.    mov dl,hour1     ;时个位
154.    mov dh,0
155.    mov si,dx
156.    mov al,[bx][si]
157.    mov dx,p8255+1   ;点亮 led
158.    out dx,al
159.    mov ah,11101111b
160.    mov al,ah
161.    mov dx,p8255+2   ;pc 输出选通信号
162.    out dx,al

```

```

163.    PUSH CX
164.    MOV CX,00f00h ;延时
165.
166. delay5:
167.    LOOP delay5
168.    POP CX
169.    mov ah,11111111b
170.    mov al,ah
171.    mov dx,p8255+2 ;pc 输出选通信号
172.    out dx,al
173.    mov dl,hour2 ;时十位
174.    mov dh,0
175.    mov si,dx
176.    mov al,[bx][si]
177.    mov dx,p8255+1 ;点亮 led
178.    out dx,al
179.    mov ah,11011111b
180.    mov al,ah
181.    mov dx,p8255+2 ;pc 输出选通信号
182.    out dx,al
183.    PUSH CX
184.    MOV CX,00f00h ;延时
185. delay7:
186.    LOOP delay7
187.    POP CX
188.    mov ah,11111111b
189.    mov al,ah
190.    mov dx,p8255+2 ;pc 输出选通信号
191.    out dx,al
192.    jmp play

```

```
193.    mov ah,4ch
194.    int 21h
195. INT_0A proc near    ;中断服务子程序
196.    cmp sec1,9
197.    jz  int1
198.    inc sec1
199.    jmp exit
200. int1:
201.    mov sec1,0
202.    inc sec2
203.    cmp sec2,6
204.    jnz exit
205.    mov sec2,0
206.    inc min1
207.    cmp min1,10
208.    jnz exit
209.    mov min1,0
210.    inc min2
211.    cmp min2,6
212.    jnz exit
213.    mov min2,0
214.    inc hour1
215.    cmp hour2,2
216.    jnz m1
217.    cmp hour1,4
218.    jnz exit
219.    jmp m2
220. m1:cmp hour1,10
221.    jnz exit
222. m2:inc hour2
```

```
223.    cmp hour2,3
224.    jnz exit
225.    mov sec1,0
226.    mov sec2,0
227.    mov min1,0
228.    mov min2,0
229.    mov hour1,0
230.    mov hour2,0
231.
232. exit:
233.    mov al,20h
234.    out 20h,al
235.    iret
236. INT_0A endp
237. code ends
238. end start
```

4. 实验结果

按照讲义中实验硬件连接方法将硬件连接好，启动 TPC-ZK-II 通用微机接口实验系统，指示灯全部正常亮起，如图 4-1 所示：

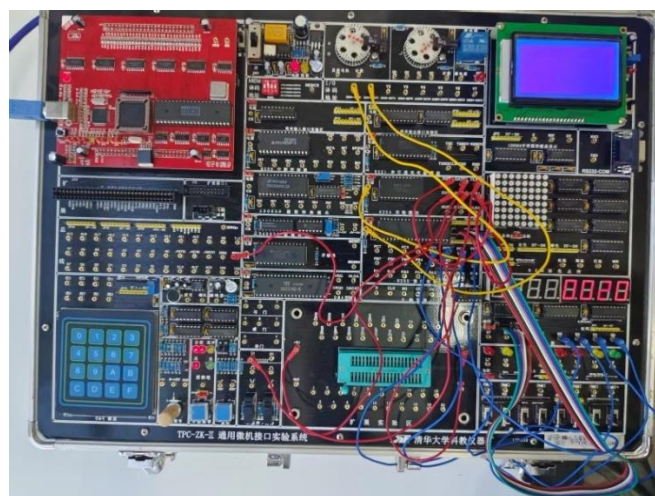


图 4-1 工作中的 TPC-ZK-II 通用微机接口实验系统

打开 TPC-ZK-II 集成开发环境，选择“选项”中的“硬件检测”，提示硬件已连接，证明设备无误，可以开始进行实验。

进行与实验二相同的一系列软件操作后，程序成功运行并弹出运行窗口。

观察实验系统右侧的数码管，可以看到，数码管一开始显示为“5955”，与程序中设定的初始时间为 23: 59: 55 相同，证明初始时间显示正确。

等待几秒后，观察数码管的变化，如下图 4-2 所示：

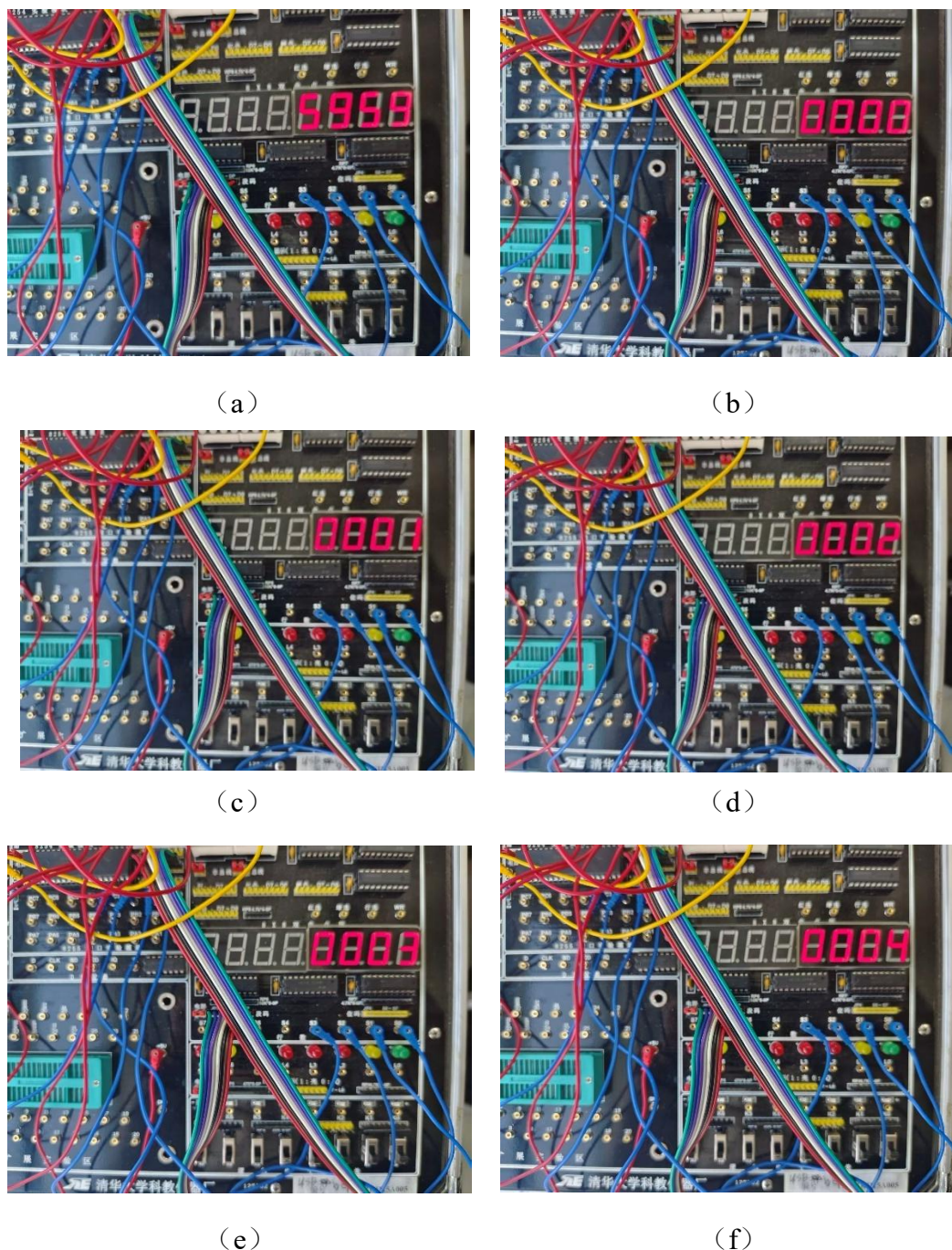


图 4-2 随时间推移数码管的变化

观察到，大约每隔 1 秒，数码管显示的数字增加 1，并且“5959”的下一时刻为“0000”，符合时钟计数原理，同时证明 4 位数码管都可以正常变化。

至此，本小组实验三的输出结果完全正确，证明此实验的硬件连接、软件程序是正确的。

5. 实验分析 ()

本次《使用 8259 中断实现定时/计数器实验》是一次综合性极强的系统设计实践。我们成功构建了一个基于中断驱动的数字时钟系统，将 8254 定时器、8259 中断控制器和 8255 并行接口有机地结合在一起。以下是我对本次实验的全面分析与总结：

1) 系统级设计思想与核心机制理解

本实验的精髓在于理解了“定时产生中断，中断驱动更新，接口控制显示”这一完整的闭环系统工作流程。与实验二的纯查询方式不同，本实验采用了中断这一高效的异步事件处理机制。CPU 无需持续轮询外部状态，而是由 8254 定时器在硬件层面精确地定时，并通过 8259 中断控制器主动通知 CPU 进行处理，极大地提高了 CPU 的工作效率。这种从主动查询到被动响应的思维转变，是掌握现代计算机系统工作原理的关键一步。

2) 8254 定时器

8254 是本系统的时间基准。我们对其两个通道进行了精确的级联配置：

- 通道 0：工作在方式 3，计数初值设置为 10000。由于 CLK0 接入 1MHz 时钟，因此 OUT0 输出的方波周期为 10ms。这成为了整个系统的基本时间单位。
- 通道 1：工作在方式 3，计数初值设置为 100。它将通道 0 输出的 10ms 信号作为其时钟输入，因此 OUT1 输出的方波周期为 1 秒。这个 1 秒的信号被连接到 IRQ3，作为每秒一次的中断请求源。

通过这种级联配置，我们利用 8254 将高频的 1MHz 时钟分频成了精确的 1 秒中断，充分展示了可编程定时器的灵活性和精确性。

3) 8259 中断控制器

8259A 负责管理来自外部的硬件中断。我们的核心操作是接管并重新编程了

系统的 IRQ3 中断。

- 中断向量设置：通过 DOS 的 35H 和 25H 功能调用，我们首先保存了系统原有的 0BH 号中断向量，然后将我们自己的中断服务程序 INT_0A 的入口地址写入中断向量表。这样，当 IRQ3 中断发生时，CPU 会自动跳转到我们的服务程序执行。
- 中断屏蔽字操作：我们读取了 8259 的原始中断屏蔽字，在确保不破坏其他中断设置的前提下，使用 AND AL, 11110111b 指令单独开放了 IRQ3 的中断请求。这一步是中断能够被 CPU 响应的前提，体现了对系统资源协同工作的考量。

4) 中断服务程序

中断服务程序 INT_0A 是整个数字钟的大脑。我深入分析了其逐级进位的时间逻辑：

- 秒个位 (sec1) 计数：每次中断（每秒一次）触发，sec1 加 1。计满 9 后归零，并向 sec2（秒十位）产生进位。
- 秒十位 (sec2) 计数：收到进位后加 1，计满 6（即 59 秒）后归零，并向 min1（分个位）产生进位。
- 以此类推：程序严格按照 60 秒进 1 分、60 分进 1 时、24 小时归零的时钟逻辑进行设计。特别值得注意的是对小时位的处理：当 hour2（时十位）为 2 时，hour1（时个位）计满 4（即 24 点）即归零，否则计满 10 归零。这段代码逻辑严密，确保了时间显示的绝对正确。

5) 数码管动态显示与软件调试技巧

显示部分采用了经典的动态扫描技术。由于 8255 的 PB 口需要为所有数码管提供段码，我们通过 PC 口循环输出位选信号，并利用人眼的视觉暂留效应，在极短的时间内依次点亮各个数码管，从而呈现出同时显示的效果。

- 延时的重要性：在每次输出位选信号后，程序都插入了一段软件延时。这个延时至关重要：如果太短，数码管亮度会不足；如果太长，则会出现明显的闪烁。我们通过调整循环次数 CX=00f00h，找到了一个亮度与稳定性俱佳的平衡点。
- 现场保护：在中断服务程序开始，系统自动保护了 FLAGS、CS 和 IP。

在显示程序的延时循环中，我们也注意了对 CX 等寄存器的压栈和出栈保护，避免了程序执行流程的混乱。

6) 总结

通过本实验，我不仅掌握了 8254、8259、8255 三大可编程接口芯片的联合编程技术，更重要的是建立了基于中断的实时系统的设计理念。我从硬件连接、初始化编程、中断向量处理，到服务程序逻辑编写和显示驱动，完整地实现了一个小型嵌入式系统。对于中断的“现场保护与恢复”、“中断嵌套”、“EOI 命令”等概念有了从理论到实践的深刻认知。这次成功的实验经历，为我今后从事更复杂的嵌入式系统和实时操作系统开发打下了坚实的基础。

6. 实验分析 ()

在使用 8259 中断实现定时/计数器实验中，我们小组以构建中断驱动的数字时钟系统为核心目标，完成了从 8254 定时器级联配置、8259 中断控制器编程、中断服务程序编写，到 8255 并行接口驱动数码管显示的实验过程。不仅实现了分秒实时计数和数码管动态显示的基础功能，更成功将 8254、8259、8255 三大可编程接口芯片有机结合在一起，让我们对中断驱动的实时系统的工作机制与软硬件协同逻辑有了更深刻的理解。以下从技术角度对本次实验展开详细分析与总结：

1) 实验设计思想和关键逻辑

相较于实验二，本实验不再依靠纯查询的操作方式，在实验过程中通过定时器产生中断信号，再由中断服务程序驱动时间更新，最后通过接口控制数码管显示，这一过程正是实时嵌入式系统的典型设计思路。

与查询方式中 CPU 需持续查询外部设备状态不同，本实验通过 8259 中断控制器采用中断异步处理的机制。8254 定时器在硬件层面实现精确计时，当计时周期到达时，通过 8259 中断控制器主动向 CPU 发送中断请求。CPU 收到请求后，暂停当前主程序，自动跳转到中断服务程序处理时间更新。更新完成后返回主程序，由 8255 接口驱动数码管显示最新时间。

这种 CPU 被动响应中断的方法，彻底释放了 CPU 资源，避免了 CPU 无效查询造成的效率浪费。而从主动查询到被动响应的方式转变，也让我们懂得了现

代计算机系统处理异步事件的核心逻辑,从而对实时系统的设计有了更深刻的理解和体会。

2) 8259 中断控制器

8259A 负责管理来自外部的硬件中断,其核心任务是将 8254 产生的 1 秒中断请求 (IRQ3) 正确传递给 CPU,并确保不干扰系统原有中断资源。我们的编程操作主要围绕中断向量配置与中断屏蔽控制两方面展开:

- 中断向量的保存与重写。系统默认 IRQ3 对应 0BH 中断类型号,为避免覆盖原中断功能,实验中先通过 DOS 的 35H 功能调用读取原 0BH 中断向量,并将其保存到变量 Old_0A 中;随后通过 25H 功能调用,将自定义的中断服务程序 INT_0A 的入口地址写入中断向量表。这样,当 IRQ3 中断触发时,CPU 会自动跳转到 INT_0A 执行,而非系统默认程序。
- 中断屏蔽字的精确控制。为确保 IRQ3 中断能被 CPU 响应,同时不影响其他中断,我们先通过 IN AL, p8259+1 指令读取 8259 奇地址寄存器中的原始中断屏蔽字,保存到变量 Old_8259 中;再通过 AND AL, 11110111b 指令将屏蔽字的第 3 位(对应 IRQ3)置 0,其余位保持不变——该操作仅开放 IRQ3 中断,不破坏系统原有中断屏蔽设置,体现了对系统资源协同使用的严谨考量。

3) 8254 定时器

8254 定时器作为整个数字时钟的时间基准源,其主要作用是将实验板上 1MHz 的高频时钟信号分频为精确的 1 秒中断信号,实验中我们通过双通道级联的方法实现这一功能,具体配置方法如下:

- 通道 0 配置。设定为工作方式 3(方波发生器),计数初值设置为 10000。由于通道 0 的 CLK0 引脚直接接入实验板 1MHz 时钟信号,由于周期=计数初值/输入时钟频率,通道 0 输出方波的周期为 $10000/1000000=10\text{ms}$ 。这一 10ms 信号成为整个系统的基本时间单位,为后续实现秒级计时奠定基础。
- 通道 1 配置。同样设定为工作方式 3,计数初值设置为 100。通道 1 的 CLK1 引脚不接外部时钟,而是连接通道 0 的 OUT0 引脚,即把通道 0 输出的 10ms 信号作为自身输入时钟。由此计算,通道 1 输出方波的周

期为 $100 \times 10\text{ms} = 1\text{s}$ 。

最终，通道 1 的 OUT1 引脚连接到系统总线的 IRQ3 引脚，将这一 1 秒周期的方波信号作为每秒一次的中断请求源，为数字时钟的秒计数提供精准触发信号。这种级联配置充分体现了可编程定时器的灵活性，实现了从高频时钟到实用计时信号的高效转换。

4) 中断服务程序的时间进位控制

中断服务程序 INT_0A 是数字时钟最关键的程序，其主要功能是实现秒→分→时的逐级时间进位，确保时间计数与真实时钟规则相符。程序中通过 sec1（秒个位）、sec2（秒十位）、min1（分个位）、min2（分十位）、hour1（时个位）、hour2（时十位）六个变量记录时间，具体进位逻辑如下：

- 秒个位计数。每次中断（每秒一次）首先让 sec1 加 1，若 sec1 未计满 9，则直接退出服务程序；若 sec1 计满 9，则将 sec1 归零，并向 sec2 产生进位。
- 秒十位计数。sec2 收到进位后加 1，若 sec2 未计满 6，则退出程序；若 sec2 计满 6（即 59 秒结束），则将 sec2 归零，并向 min1 产生进位。
- 分位计数逻辑。与秒位完全一致，仍然是 min1 满 9 归 0 向 min2 产生进位，min2 满 6 归 0 向 hour1 产生进位。
- 小时位特殊处理。根据实际时钟情况并考虑到一天只有 24 小时，程序对小时位进位进行了特殊处理：若 hour2 为 2，即时间处于 20~23 时，则 hour1（计满 4 后，将 hour1、hour2 一同归零；若 hour2 不为 2，则 hour1 满 10 归 0 进 hour2。

中断服务程序的这段时间进位代码遵循 60 秒进 1 分、60 分进 1 时、24 小时归零的时钟规则，确保了时间计数的准确性，是数字时钟功能实现的关键所在。

5) 数码管显示方法与汇编程序调试细节

数字时钟的可视化输出可通过 8255 驱动数码管动态扫描实现，核心是利用人眼视觉暂留效应，让多个数码管同时发光，具体实现与调试细节如下：

- 硬件与接口对应。8255 的 PB 口（PB7~PB0）连接数码管的 dp~a 引脚，负责输出段码，即控制单个数码管显示的数字；PC 口（PC3~PC0）连接数码管的 S3~S0 引脚，负责输出位选信号，控制哪个数码管点亮。程序

中预先定义了数码管段码表 `led`，为显示提供数据支撑。

- 动态扫描逻辑。程序通过循环依次点亮每个数码管，先从时间变量中获取当前要显示的数字，从段码表中匹配对应的段码并通过 `PB` 口输出；再通过 `PC` 口输出对应的位选信号，让目标数码管通电；随后插入软件延时，延时结束后关闭当前数码管，再切换到下一个数码管重复操作。
- 关键调试细节。首先要确定恰当的延时时间，延时时间直接影响显示效果，若延时太短，数码管亮度不足；若延时太长，会出现明显闪烁。实验中通过反复测试，将延时循环的 `CX` 值设定为 `00f00h`，最终找到数码管亮度充足且无明显闪烁的延时时间。与此同时实验中还需要注意现场保护，在主程序的延时循环中，通过 `PUSH CX` 和 `POP CX` 指令保护 `CX` 寄存器值，避免延时操作破坏其他程序逻辑；而在中断服务程序中，系统会自动保护 `FLAGS`、`CS`、`IP` 寄存器，确保中断返回后主程序能正常执行，这些操作是避免程序流程混乱的关键。

6) 实验总结

在本次使用 8259 中断实现定时/计数器实验中，我学会了使用多种芯片进行系统级设计。在实验过程中，我不仅掌握了 8254 定时器级联分频、8259 中断向量配置与屏蔽控制、8255 驱动数码管动态显示的具体方法，更重要的是建立了中断驱动的实时系统设计理念，理解了实验中“硬件定时触发中断、中断驱动软件更新、软件控制硬件显示”的核心思想，也对中断现场保护与恢复、EOI 命令、中断嵌套等抽象概念有了具体的认识。

从硬件连接时 8254 `OUT1` 与 `IRQ3` 的精准对接，到软件编程时中断服务程序的进位逻辑设计，再到调试时数码管延时参数的反复优化，每一步都让我深刻体会到实时系统中软硬件需高度协同的工程原则。这次实验不仅夯实了我对三大可编程接口芯片的应用能力，更为后续学习复杂嵌入式系统、实时操作系统开发打下了坚实基础，也让我对如何设计高效、稳定的实时控制方案有了更深刻的思考。

7. 实验分析 ()

此实验将 8254 定时器、8259 中断控制器与 8255 并行接口相结合，构建了一个基于中断驱动的数字时钟系统。通过实践，我不仅掌握了多个可编程芯片的联

合编程技巧，更对中断机制在实时系统中的应用有了深刻理解。

与前序实验仅涉及单一芯片不同，本实验需要三个核心芯片协同工作：8254 提供精确定时，8259 管理中断请求，8255 控制数码管显示。每个芯片承担独立功能，通过硬件连接和软件初始化形成闭环系统。8254 的通道 0 将 1MHz 时钟分频为 10ms 信号，通道 1 再将其分频为 1 秒信号，这个 1 秒信号作为中断请求输入 8259 的 IRQ3；8259 响应中断后通知 CPU 执行中断服务程序，更新时间变量；最后通过 8255 将时间数据输出到数码管显示。

其中，8254 的级联配置是实现 1 秒计时的关键。通道 0 工作在方式 3（方波模式），输入时钟 1MHz，要输出 10ms 周期的信号，计数初值为 $1\text{MHz} \times 10\text{ms} = 10000$ ；通道 1 同样工作在方式 3，输入时钟为通道 0 的输出（10ms 周期），要输出 1 秒周期的信号，计数初值为 $1\text{秒} / 10\text{ms} = 100$ 。编程时，要按照先写控制字，再写低 8 位计数初值，后写高 8 位计数初值的顺序配置，否则会导致定时不准确。

中断控制器 8259 的编程是实验的难点之一。首先通过 DOS 的 35H 功能调用保存原 0BH 中断向量，避免破坏系统原有中断；然后通过 25H 功能调用将自定义的中断服务程序 INT_0A 的入口地址写入中断向量表；最后修改 8259 的中断屏蔽字，单独开放 IRQ3 中断。在中断服务程序中，我遵循先关中断、更新时间、发送 EOI 命令、开中断的流程，确保中断响应的正确性。特别是 EOI 命令的发送，若遗漏会导致 8259 无法响应后续中断，时钟停止更新。

数码管采用动态扫描方式显示，通过 8255 的 PB 口输出段码、PC 口输出位选信号。由于人眼的视觉暂留效应，只要快速循环点亮各个数码管，就能呈现出同时显示的效果。延时时间的设置至关重要：延时过短，数码管亮度不足；延时过长，会出现明显闪烁。通过调整延时循环的计数初值（CX=00f00h），我们最终实现了清晰稳定的显示。同时，在编程时需要注意寄存器的现场保护，避免延时循环破坏其他寄存器的值，这也让我养成了一个严谨的编程习惯。

本次实验是对我系统设计能力的一次全面锻炼。从硬件连接时各芯片引脚的对应，到软件中各模块的初始化、中断向量的处理、时间逻辑的编写，每个环节都需要细致的思考和调试。通过解决实验中遇到的问题，我不仅掌握了 8254、8259、8255 的编程技巧，更提升了问题排查和系统优化的能力。

8. 实验分析 ()

1) 软硬件协同构建实时系统

本实验是计算机组成原理中最典型的“实时系统”雏形。我们成功地将 8254 定时器、8259 中断控制器和 8255 并行接口结合在一起。

- 时间基准的产生：系统的时间精度完全依赖于 8254 的级联配置。通道 0 作为预分频器，将 1MHz 的基准时钟分频 10000（计数初值 10000），产生 100Hz 的信号 5。通道 1 进一步分频 100（计数初值 100），最终在 OUT1 端输出了精确的 1Hz（1 秒）方波 6。这让我理解了在嵌入式系统中如何通过硬件计数器获取精准的系统心跳。

2) 中断系统的接管与现场保护

实验中最关键的软件操作是对中断向量表的修改。

- 接管过程：我们没有直接覆盖中断向量，而是先使用 DOS 功能调用 35H 读取并保存了系统原有的 0BH 号中断向量 7，然后才使用 25H 设置新的中断向量 8。这是编写系统级程序（如驱动程序或驻留程序）必须遵守的规范，确保程序退出后系统能恢复正常，防止崩溃。
- 中断服务：在 INT_0A 子程序中，我深入理解了“逻辑与硬件分离”的思想。硬件（8254+8259）只负责每秒产生一次触发信号，而真正的时钟逻辑（秒进位、分进位、24 小时清零）完全由软件中的 CMP 和 INC 指令实现 9。

3) 动态扫描显示的视觉暂留原理

实验中数码管的显示采用了动态扫描技术。

- 实现机制：由于 8255 端口资源有限，不能同时维持所有数码管的段选信号。程序通过快速轮流选中位选（由 PC 口控制），并同步送出对应的段码（由 PB 口控制）。
- 延时的权衡：代码中的 delay 循环至关重要 10。如果延时过短（循环次数少），数码管亮度不足；如果延时过长，人眼就会察觉到闪烁。实验中设置的 0F00H 循环次数 11 恰好利用了人眼的视觉暂留效应，实现了稳定的静态显示效果。

实验四 使用 8251A 的串行接口应用实验

1. 实验目的

- 1) 掌握串行通信原理及半双工和全双工的编程方法;
- 2) 掌握用 8251A 接口芯片实现微机间的同步和异步通信;
- 3) 掌握 8251A 芯片与微机的接口技术和编程方法。

2. 实验原理和内容

8251A 是一种可编程的同步/异步串行通信接口芯片, 具有独立的接收器和发送器, 能实现单工、半双工、双工通信。

1) 8251A 内部结构

8251A 通过引脚 D0~D7 和系统数据总线直接接口, 用于和 CPU 传递命令、数据、状态信息。读写控制逻辑用来接收 CPU 的控制信号、控制数据传送方向。CPU 对 8251A 的读写操作控制表如表 2-1 所示。

表 2-1 CPU 对 8251A 的读写操作控制表

/CS	C/D	/RD	/WR	操作
1	任意	任意	任意	无操作,D0-D7 呈高阻
0	1	1	0	写控制字
0	0	1	0	写数据
0	1	0	1	读状态
0	0	0	1	读数据

2) 8251A 的方式控制字和命令控制字

方式控制字确定 8251A 的通信方式(同步/异步)、校验方式(奇校/偶校/不校)、字符长度及波特率等, 格式如图 2-1 所示。

命令控制字使 8251A 处于规定的状态以准备收发数据, 格式如图 2-2 所示。

方式控制字和命令控制字无独立的端口地址, 8251A 根据写入的次序来区分。

CPU 对 8251A 初始化时先写方式控制字, 后写命令控制字。

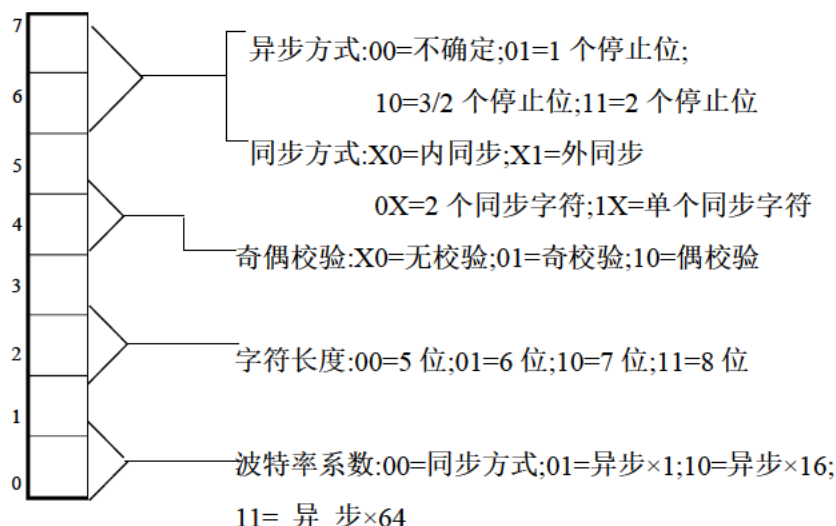


图 2-1 8251 方式控制字

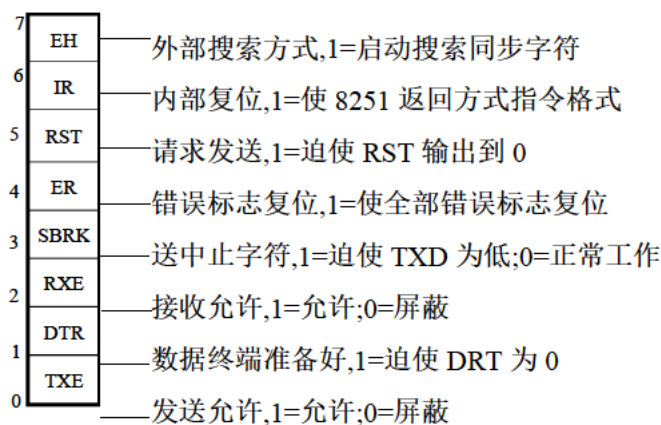


图 2-2 8251 命令控制字

3) 状态寄存器

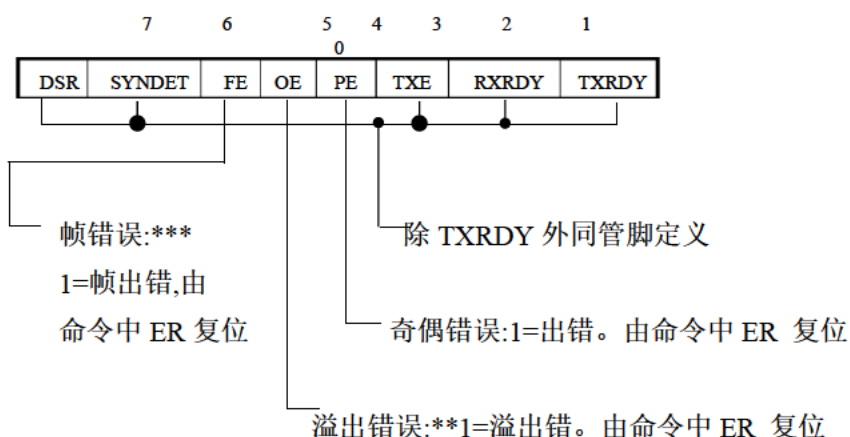


图 2-3 状态寄存器

8251 状态寄存器用于寄存 8251A 的状态信息，供 CPU 查询，定义如图 2-3

所示。

- TXRDY 位：当数据缓冲器空时置位，而 TXRDY 引脚只有当条件(数据缓冲器空•/CTS•TXE)成立时才置位。
- 溢出错误：CPU 没读走前一个字符，下一个字符又接收到，称为溢出错误。
- 帧错误：在字符结尾没检测到停止位，称为帧错误。

4) PC 机寄存器的端口地址

表 2-2 PC 机寄存器的端口地址

地址	寄存器名称	备注说明
0x3F8	发送保持寄存器（THR）	DLAB=0
0x3F8	接受缓存寄存器（RBR）	DLAB=0
0x3F8	波特率因子寄存器[低]（DLL）	DLAB=1
0x3F9	波特率因子寄存器[高]（DLM）	DLAB=1
0x3F9	中断允许寄存器（IER）	DLAB=0
0x3FA	中断识别寄存器（IIR）	
0x3FB	线路控制寄存器（LCR）	
0x3FC	MODEM 控制寄存器（MCR）	*
0x3FD	线路状态寄存器（LSR）	
0x3FE	MODEM 状态寄存器（MSR）	*

其中，线路控制寄存器第七位：DLAB=0；线路控制寄存器第七位：DLAB=1。

5) 波特率和除数因子对照表

表 2-3 波特率和除数因子对照表

波特率 单位 HZ	因子值		波特率 单位 HZ	因子值	
	MSB	LSB		MSB	LSB
50	09	00	1800	00	40
75	06	00	2000	00	3A
110	04	17	2400	00	30
134. 5	03	59	3600	00	20
150	03	00	4800	00	18
300	01	80	7200	00	10
600	00	C0	9600	00	0C
1200	00	60			

实验硬件连接方法：通过计算机点到点三线制串口通信线，掌握接线的方法以及 RS-232 标准的机械、电气规范。

3. 实验程序

基于 8251 芯片实现异步串行通信一般有两种方式，一种是查询方式，另一种是中断方式，使用哪种方式取决于进行初始化时寄存器的设置。

8254 计数器的计数初值=时钟频率/(波特率×波特率因子)，这里的时钟频率接 1MHz，波特率若选 1200，波特率因子若选 16，则计数器初值为 52。

本实验采用 8251A 异步方式发送，利用 8086 汇编语言实验计算机点到点的串口通信，设置：波特率为 1200bps、数据位 7 位、停止位 1 位、偶校验方式，利用查询方式或中断方式实现双机通信，能够传输多个字符。

设计发送方和接收方两个程序，要求将某汇编语言传送到接收方，接收方收到后将源程序写入指定磁盘或屏幕显示。

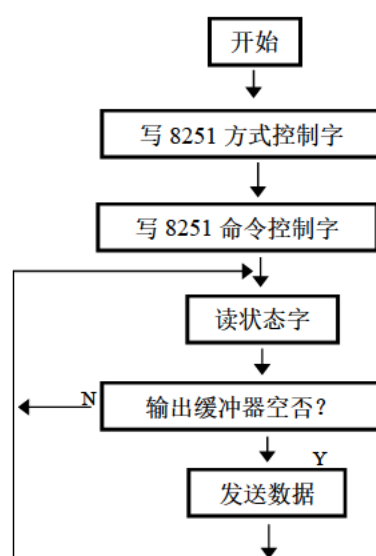


图 3-1 串行接口通信编程流程图

具体程序如下：

```
1. data segment
2. io8253a equ 280h
3. io8253b equ 283h
4. io8251a equ 2b8h
5. io8251b equ 2b9h
6. mes1 db 'you can play a key on the keyboard! ',0dh,0ah,'$'
7. mes2 dd mes1
8. data ends
```

```
9. code segment
10. assume cs:code,ds:data
11. start:
12.     mov ax,data
13.     mov ds,ax
14.     mov dx,io8253b
15.     mov al,16h
16.     out dx,al
17.     mov dx,io8253a
18.     mov al,52
19.     out dx,al
20.     mov dx,io8251b
21.     xor al,al
22.     mov cx,03
23. delay:
24.     call out1
25.     loop delay
26.     mov al,40h
27.     call out1
28.     mov al ,05ah
29.     call out1
30.     mov al,27h
31.     call out1
32.     lds dx,mes2
33.     mov ah,09h
34.     int 21h
35. waiti:
36.     mov dx,io8251b
37.     in al,dx
38.     test al,38h
```

```
39.    jnz waiti
40.    test al,01
41.    jz waiti
42.    mov ah,01h
43.    int 21h
44.    cmp al,27
45.    jz exit
46.    mov dx,io8251a
47.    out dx,al
48.    mov cx,40h
49. s51:
50. loop s51
51. next:
52.    mov dx,io8251b
53.    in al,dx
54.    test al,02
55.    jz next
56.    mov dx,io8251a
57.    in al,dx
58.    mov dl,al
59.    mov ah,02
60.    int 21h
61.    mov dl,20h
62.    mov ah,02h
63.    int 21h
64.    jmp waiti
65. exit:
66.    mov ah,4ch
67.    int 21h
68. out1 proc near
```

```
69.    out dx,al
70.    push cx
71.    mov cx,40h
72. gg:
73.    loop gg
74.    pop cx
75.    ret
76.out1 endp
77. code ends
78. end start
```

4. 实验结果

按照讲义中实验硬件连接方法将硬件连接好，启动 TPC-ZK-II 通用微机接口实验系统，指示灯全部正常亮起，如图 4-1 所示：

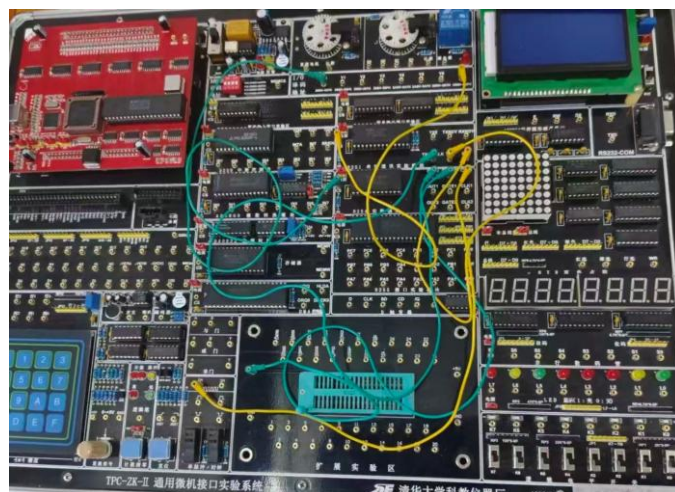


图 4-1 工作中的 TPC-ZK-II 通用微机接口实验系统

打开 TPC-ZK-II 集成开发环境，选择“选项”中的“硬件检测”，提示硬件已连接，证明设备无误，可以开始进行实验。

进行与实验二相同的一系列软件操作后，程序成功运行并弹出运行窗口，等待输入。

从电脑键盘输入不同的符号，观察运行窗口的输出。

首先，输入符号“k”，观察运行窗口的变化，结果为屏幕上输出了“k”，如

图 4-2 所示：

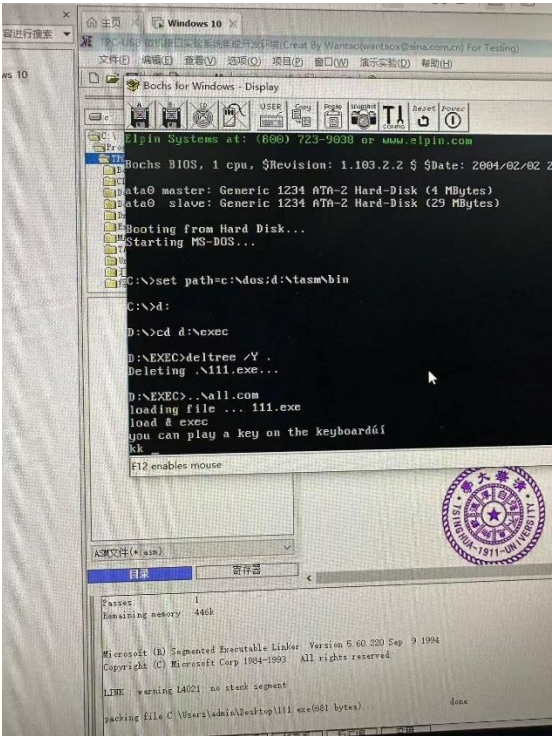
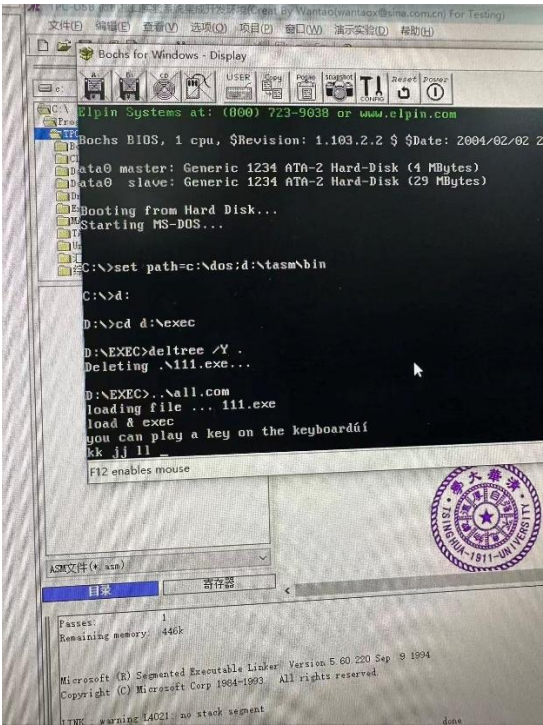
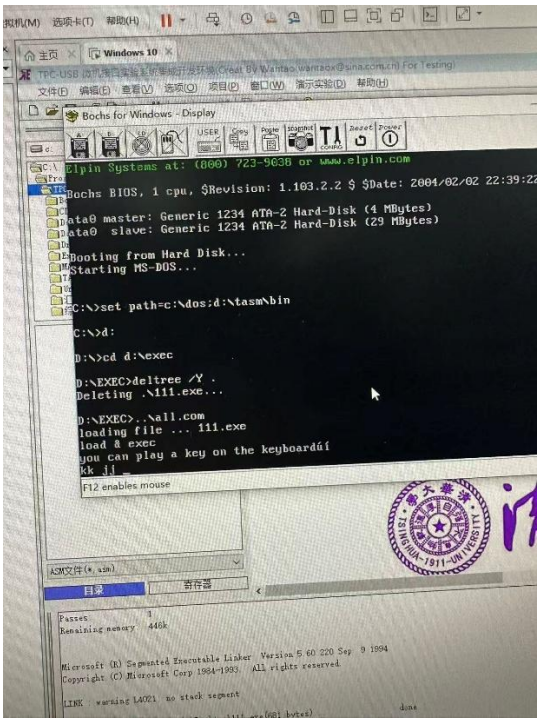


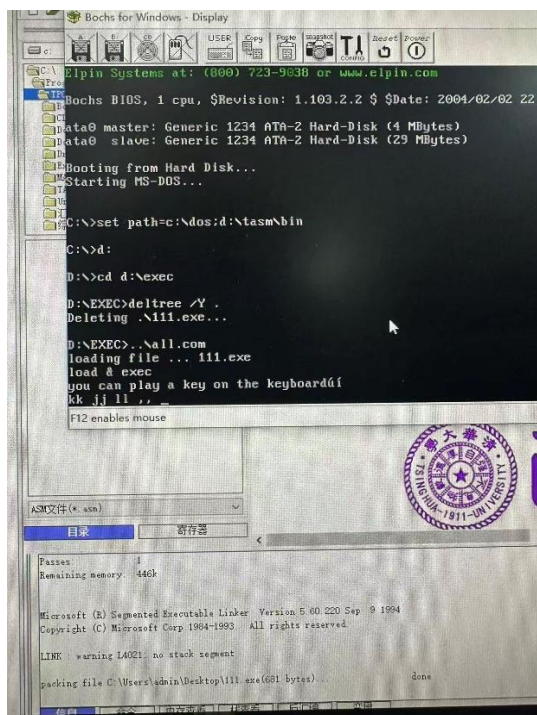
图 4-2 从电脑键盘输入符号“k”的输出结果

可以看到，发送方发送符号“k”，接收方期待接收到符号“k”并显示，故结果是正确的。

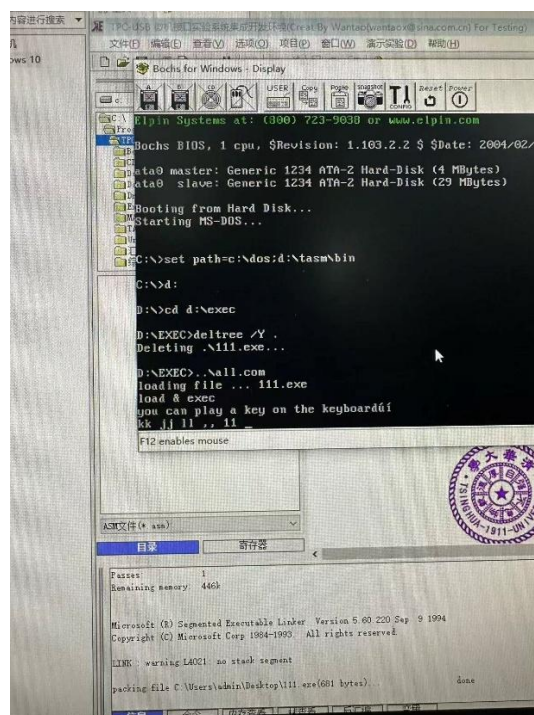
为了验证传送的任意性，接着输入符号“j”、“l”（字母）、“，”、“1”（数字），观察运行窗口的变化，结果如图 4-3 所示：



(a)



(b)



(c)

(d)

图 4-3 输出结果

可以看到，发送方发送任意符号，接收方接收到相同符号并显示。

至此，本小组实验四的输出结果完全正确，证明此实验的硬件连接、软件程序是正确的。

5. 实验分析 ()

本次《使用 8251A 的串行接口应用实验》成功地将计算机系统的数据交换能力从机内并行扩展到了机间串行通信领域。我们构建了一个基于 8251A 芯片的异步串行通信系统，实现了两台设备间的点对点数据收发。以下是我对本次实验的全面分析与总结：

1) 串行通信核心思想与 8251A 作用理解

与实验二、三中涉及的并行接口不同，本实验的核心在于串行通信。我深刻理解了串行通信“将数据字节拆分为位，在单条信道中按时间顺序依次传输”的基本思想。这种方式虽然单位时间内的数据传输率低于并行通信，但其优势在于极大的简化了线路连接，特别适用于远距离通信。8251A 作为可编程通信接口芯

片，正是实现这一思想的关键。它承担了并/串转换和串/并转换的核心任务，并自动处理了包括帧格式封装、校验位生成与检测、波特率控制等底层通信细节，使 CPU 能够以类似并行端口的方式操作串行通信。

2) 系统初始化流程的精确设计与实现

8251A 的初始化是实验成功的首要环节，其过程较为严格，我深入理解并实现了这一流程：

- 8253 定时器配置：串行通信的时序基础是精确的波特率。我们使用 8253 定时器通道 2 来产生 8251A 所需的发送和接收时钟。根据公式“计数初值=输入时钟频率/(目标波特率×波特率因子)”，我们设定输入时钟为 1MHz，目标波特率为 1200bps，波特率因子为 16，从而计算出计数初值为 52。将此值写入 8253 通道 2，就为 8251A 提供了精确的基准时钟。
- 8251A 复位与方式字设置：8251A 的初始化需要一个软复位过程。我们通过先向控制口写入 3 个 0，再写入 40H 来实现。随后，写入方式控制字 5AH，该字的每一位都经过精心设计：
 - ✓ D1D0=10：设定波特率因子为 16 倍。
 - ✓ D3D2=01：设定字符长度为 7 位。
 - ✓ D5D4=01：选择偶校验。
 - ✓ D7D6=01：设定 1 位停止位。

这个控制字 01011010B 完整定义了本次通信的异步数据帧格式。

- 8251A 命令字设置：最后写入命令控制字 27H，其关键位设置为：
 - ✓ D0=1(TXEN)：允许发送。
 - ✓ D1=1(DTR)：通知对方设备本机已就绪。
 - ✓ D2=1(RXE)：允许接收。

这个命令字 00100111B 使 8251A 进入工作状态，准备收发数据。

3) 查询式通信机制与程序流程剖析

本实验采用查询方式进行通信，我详细分析了其工作循环：

- 发送流程：程序首先查询状态寄存器的 TXRDY 位。当该位为 1 时，表示 8251A 的发送缓冲器已空，可以接收 CPU 送来的新数据。此时，CPU 将从键盘读取的字符通过数据端口写入 8251A，由 8251A 自动完成并串

转换和帧格式封装后发送出去。

- 接收流程：紧接着，程序查询状态寄存器的 **RXRDY** 位。当该位为 1 时，表示 8251A 已成功接收一个完整的字符并完成了串并转换，数据已存入接收缓冲器。CPU 随即从数据端口读取该字符，并通过 DOS 功能调用在屏幕上显示。
- 错误处理：在发送前，程序还查询了状态字的高三位 (**FE,OE,PE**)，分别对应帧错误、溢出错和奇偶校验错。一旦检测到错误，程序会等待错误状态清除后再进行发送，这是一种基本的通信可靠性保障机制。

4) 对通信协议与硬件协同的深入认识

通过本实验，我不仅编写了通信程序，更对 RS-232 串行通信协议有了直观的认识。我们实现的“7 位数据、偶校验、1 位停止位”是标准的异步串行通信格式。同时，我也认识到，一个完整的通信系统需要多个芯片协同工作：8253 提供精准的波特率时钟，8251A 负责通信协议的处理，而 CPU 则负责总体控制和数据搬运，这种协同设计的思想是构建复杂电子系统的基础。

5) 总结

本次实验使我成功掌握了 8251A 串行接口芯片的编程方法，并亲身体验了查询式串行通信的全过程。我从波特率计算、芯片初始化、状态查询到数据收发，完整地实现了一个双机通信系统。在实践中，我深刻体会到通信协议中每一位的重要性，以及精确时序在串行通信中的关键作用。相比于实验三的中断方式，本实验采用的查询方式虽然简单，但在实际应用中 CPU 效率较低，这让我自然联想到在需要高效率的系统中，采用中断驱动的串行通信将是更优的选择。这次实验为我今后理解和使用更复杂的通信协议乃至网络通信奠定了坚实的实践基础。

6. 实验分析 ()

在本次的使用 8251A 的串行接口应用实验中，我们小组利用 8251A 芯片构建了一个异步串行通信系统，完成了从 8253 定时器波特率时钟配置、8251A 芯片初始化编程，到查询式数据收发的汇编程序编写与硬件调试的实验任务。该实验不仅实现了键盘输入字符→双机传输→屏幕显示的基础功能，更成功将计算机数据交换能力从机内并行扩展至机间串行领域，让我们对串行通信原理、可编程

通信接口芯片应用及多硬件协同逻辑有了更系统的认知。以下从技术角度对本次实验展开详细分析与总结：

1) 串行通信的主要特点

本次实验与实验二、实验三的核心差异在于数据传输方式从并行转向串行，串行通信突破了并行通信多信道同时传多位的局限，采用单信道时序传位的模式：将 1 个数据字节拆分为 8 个二进制位，在同一条通信线路中，按照固定时间顺序依次传输，传输完成后再重组为字节。这种方式虽然因需要逐位传输导致单位时间数据率低于并行通信，但有着极其显著的优势：仅需 1 条数据线即可实现数据传输，极大简化了线路连接，尤其适合远距离数据交互传输。而 8251A 作为可编程串行通信接口芯片，正是实现串行通信的主要接口芯片，其功能完全服务于串行通信的底层需求：

- 承担并串/串并转换核心任务。接收 CPU 送来的并行数据时，8251A 自动将其转换为串行数据；接收外部串行信号时，自动将其重组为并行数据，无需 CPU 干预底层位操作；
- 自动处理通信细节。按预设规则完成帧格式封装、校验位生成与检测、波特率同步以匹配收发双方时序，让 CPU 只需像操作并行端口一样读写数据，无需关注复杂的通信协议细节。

2) 系统初始化设置

实验中的异步串行通信系统初始化流程必须按照按时钟配置→芯片复位→方式字设置→命令字设置的顺序操作，初始化设置不当很可能造成实验失败。我们结合实验需求，完成了精准的初始化设计：

- 8253 定时器的波特率时钟配置。串行通信的核心是时序同步，而正确设置时钟信号的波特率是时序同步的基础。8251A 自身无法生成时钟，需依赖外部时钟信号，实验中通过 8253 定时器通道 2 提供该信号，具体计算与配置如下：实验中设定输入时钟频率为 1MHz、目标波特率 1200bps、波特率因子 16 根据公式“计数初值=输入时钟频率/(目标波特率×波特率因子)”，从而计算出计数初值为 52，将该计数初值写入 8253 通道 2，最终生成 8251A 所需的收发时钟，确保通信时序稳定。
- 8251A 的复位与方式字设置。8251A 上电后需先执行软复位，才能接收

后续配置指令。通过向 8251A 控制端口连续写入 3 个 0，再写入复位控制字 40H，完成芯片状态清零，为配置做准备。随后写入方式控制字 5AH，该控制字完整定义异步通信帧格式，每一位均与实验需求对应：

- D1D0=10：设定波特率因子为 16，与 8253 生成的时钟频率匹配；
 - D3D2=01：设定数据位长度为 7 位，符合实验 7 位数据位+校验位的通信规则。
 - D5D4=01：选择偶校验模式，传输时自动添加偶校验位，接收时检测校验错误。
 - D7D6=01：设定停止位为 1 位，平衡通信效率与可靠性。
- 8251A 的命令字设置。在利用方式字定义通信规则后，需通过命令字让 8251A 进入工作状态：向控制端口写入命令控制字 27H，关键位设置如下：
- D0=1 (TXEN)：允许 8251A 发送数据，开启发送缓冲器；
 - D1=1 (DTR)：使 DTR 引脚输出低电平，通知接收设备“本机已就绪”；
 - D2=1 (RXE)：允许 8251A 接收数据，开启接收缓冲器；

至此，8251A 完成初始化，具备收发数据的条件。

3) 串行通信机制

本实验采用查询方式实现双机通信，核心逻辑是 CPU 通过读取 8251A 状态寄存器判断通信状态，再执行对应操作，以此形成查询工作循环。

➤ 数据发送流程

- 状态查询：CPU 向 8251A 控制端口读取状态字，检测 TXRDY 位；若该位为 1，说明发送缓冲器为空，可接收 CPU 送来的新数据；
- 数据写入：CPU 读取键盘输入的字符，再将字符写入 8251A 数据端口；
- 底层处理：8251A 自动为该字符添加起始位、偶校验位、1 位停止位，转换为串行位流后通过 TXD 引脚发送给接收方。

➤ 数据接收流程

- 状态查询：发送完成后，CPU 继续读取状态字，检测 RXRDY 位；若该位为 1，说明接收缓冲器已存入完整字符；

-
- 数据读取与显示：CPU 从数据端口读取接收的字符并在在屏幕上显示，实现接收数据可视化，便于判断实验效果。

➤ 通信错误处理

- 为保障通信可靠性，程序在发送前会检测状态字的错误标志位：
- FE 位：帧错误，接收时未检测到停止位；
- OE 位：溢出错，前一个字符未读取，新字符已接收；
- PE 位：奇偶校验错，接收字符的校验位与计算结果不符；

若检测到任意错误位为 1，程序会循环等待，直至错误状态清除再继续发送，避免错误数据传输。

4) 通信协议与硬件协同

实验不仅实现了通信功能，更让我们对串行通信协议与多硬件协同有了更深刻的理解：

- RS-232 协议的实验落地。在本次实验中我们实现的“7 位数据位+1 位偶校验位+1 位停止位”是 RS-232 串行通信协议的典型异步帧格式，其中起始位标识字符开始，7 位数据位传输有效信息，1 位偶校验位用于错误检测，1 位停止位标识字符结束，这套格式是双机异步通信的通用标准，确保不同设备间可兼容通信。
- 多芯片的协同工作逻辑。一个完整的串行通信系统并非 8251A 单独工作，而是 8253、8251A 与 CPU 的协同配合：
- 8253 是实验的时序基础：生成 8251A 所需的波特率时钟，确保收发双方时序同步；
 - 8251A 是通信执行者：处理底层位操作与协议细节，并降低 CPU 负担；
 - CPU 是整个系统的控制核心：负责初始化配置、数据读写、状态查询，统筹整个通信流程；

这三者缺一不可，若 8253 未生成时钟，8251A 无法同步；若 8251A 未处理协议，CPU 需耗费大量资源处理位操作，这体现了复杂电子系统分工明确、协同工作的核心设计思想。

5) 实验总结

本次使用 8251A 的串行接口应用实验，是一次从理论协议到实际通信的完整实践。通过实验，我不仅掌握了 8251A 的初始化编程、查询式通信流程设计，更深刻理解了串行通信时序同步、协议封装的核心原则，在实验中我们发现方式字中 1 位停止位的设置，直接影响接收方能否正确识别字符边界；8253 计数初值的微小偏差，会导致波特率不匹配，进而出现乱码。

同时，实验也让我清晰认识到不同通信方式的优劣：查询方式虽编程简单、易于调试，但 CPU 需持续查询状态字，无法执行其他任务，效率较低；而实验三采用的中断方式，可让 CPU 在无通信需求时处理其他工作，仅在有数据时响应中断，更适合高效系统。这种对比认知，为后续选择通信方案提供了依据。

此次实验不仅夯实了我对串行通信、8251A 芯片的应用能力，更重要的是建立了硬件时序→通信协议→软件控制的系统思维框架，为今后学习 USB、以太网等更复杂的通信协议，以及通信网络的理论和设计，奠定了坚实的实践基础。

7. 实验分析 ()

此次 8251A 串行接口实验，我初步掌握了机间串行通信的核心原理和实现方法。通过构建点对点异步串行通信系统，我对串行通信协议、8251A 芯片编程以及波特率控制有了更加全面的实践认知。

串行通信的核心特点是将数据字节拆分为单个比特，在一条传输线上按时间顺序依次传输。与并行通信相比，串行通信虽然传输速率较低，但极大简化了硬件连接，适用于远距离数据交换。8251A 芯片作为串行通信接口，承担了并串转换、串并转换以及通信协议处理的核心任务，让 CPU 无需关注底层的比特传输细节，只需像操作并行端口一样读写数据，这让我明白了接口芯片在计算机系统桥梁作用。

8251A 的初始化流程严格且关键，任何一步出错都会导致通信失败。我将初始化过程分为三个核心环节：首先配置 8253 定时器，为 8251A 提供精确的波特率时钟。根据公式 $\text{计数初值} = \text{输入时钟频率} / (\text{波特率} \times \text{波特率因子})$ ，输入时钟 1MHz，波特率 1200bps，波特率因子 16，计算得出计数初值为 52，通过写入 8253 通道 2 实现时钟生成；其次对 8251A 进行软复位，连续写入 3 个 0 和 1 个 40H，确保芯片处于初始状态；最后写入方式控制字 5AH 和命令控制字 27H，完成通

信参数配置。

本实验核心是通过检测 8251A 的状态寄存器判断通信状态。发送数据时，先查询状态寄存器的 TXRDY 位，当该位为 1 时表示发送缓冲器为空，可写入新数据；接收数据时，查询 RXRDY 位，当该位为 1 时表示接收缓冲器有数据，可读取数据。同时，程序还会检测状态寄存器的错误标志位，确保通信可靠性。通过跟踪程序执行流程，我发现查询方式虽然实现简单，但 CPU 需要持续轮询状态，效率较低。

串行通信的成功依赖于收发双方通信参数的严格一致。实验中，我们设定波特率 1200bps、7 位数据位、1 位停止位、偶校验，若任何一方参数不匹配，都会导致数据传输错误。

通过本次实验，我不仅掌握了 8251A 芯片的编程方法和串行通信的基本原理，更提升了跨设备通信系统的设计能力。从硬件连接时的 RS-232 标准接线，到软件中的波特率计算、芯片初始化、状态查询、数据收发，每个环节都需要兼顾硬件特性和软件逻辑。调试过程中，我学会了通过观察状态寄存器的值定位通信故障。同时，我也认识到查询式通信的局限性，未来可以尝试采用中断方式优化，进一步提高 CPU 效率。本次实验也为我今后学习更复杂的通信协议奠定了坚实的基础。

8. 实验分析 ()

1) 异步通信协议的底层实现

本实验让我从寄存器级别理解了 RS-232 串行通信的建立过程。与并行通信不同，串行通信必须严格约定双方的协议。

- 初始化序列：8251A 的初始化顺序极其严格。必须先进行软件复位（写入 3 个 0 和 1 个 40H），确保芯片处于已知状态 12。
- 协议配置：方式字 5AH (01011010B) 的设置直接定义了通信“语言”：1 位停止位、偶校验、7 位数据位，以及最重要的 16 倍波特率因子 13。
- 波特率发生器：8253 定时器在这里扮演了幕后英雄的角色。为了实现 1200bps 的通信速率，8253 必须提供 $1200 \times 16 = 19.2\text{kHz}$ 的时钟。我们计算出的计数初值 $1\text{MHz} / 19200 \approx 52$ 14，精确地满足了这

一时序要求。

2) 查询方式 (Polling) 的工作机制

实验采用了查询方式进行数据收发，这让我直观体会到了 CPU 与外设同步的过程。

- 状态寄存器：代码中通过 `IN AL, DX` 读取状态口，并用 `TEST AL, 01H` (`TXRDY`) 和 `TEST AL, 02H` (`RXRDY`) 来检查外设状态 15。
- 效率思考：在 `waiti` 标号处，CPU 处于“死等”状态，不断轮询状态位。这种方式虽然逻辑简单、易于调试，但在多任务环境下会极大地浪费 CPU 资源。这与实验三的中断方式形成了鲜明对比，让我深刻体会到在高性能系统中，中断驱动通信（当缓冲区空或满时才通知 CPU）是更优的选择。

3) 数据传输的完整性验证

通过双机通信实验，我验证了数据的完整链路：键盘输入 -> CPU 读取 -> 8251A 并串转换 -> 传输线 -> 接收端 8251A 串并转换 -> CPU 读取 -> 屏幕显示。

- 错误检测：虽然本次实验环境较理想，未出现传输错误，但代码中对状态字高位的读取实际上包含了对 FE (帧错误)、OE (溢出错误)、PE (奇偶校验错误) 的潜在检测能力。理解这些错误标志位对于开发健壮的工业通信程序至关重要。

总结与感谢

本次计算机原理与应用实验，是我们小组在 [REDACTED] 期间，围绕汇编语言编程、可编程接口芯片应用展开的一次系统性实践。从实验一的汇编语言基础到实验四的串行通信，我们四人依托 TPC-ZK-II 通用微机接口实验系统，完成了四个综合设计类实验，不仅达成了所有预设目标，更在软硬件协同实践中深化了对计算机底层逻辑的理解。

回顾整个实验过程，每一个环节都充满了对知识的验证与探索。实验一从汇编语言的寻址方式调试入手，我们逐行跟踪指令执行，观察寄存器与内存数据的变化，亲手实现了跨数据段的数据搬移——当看到“COMPUTER”字符串成功从 DATA 段复制到 EDATA 段时，我第一次真切感受到汇编语言与硬件内存的直接关联，也意识到哪怕一个指令格式的偏差，都可能导致整个程序逻辑失效。实验二聚焦 8255A 并行接口，我们反复核对硬件连线，将 PA 口、PB 口与小键盘、数码管精准对接，在解决“单次按键多次显示”的抖动问题时，通过分析扫描周期与机械按键特性，理解了软件消抖的必要性，也体会到硬件特性对软件逻辑的直接影响。实验三的中断定时实验则是一次系统级的挑战，我们需要同时配置 8254 定时器、8259 中断控制器与 8255 显示接口，从级联定时器生成 1 秒中断信号，到修改中断向量表、编写中断服务程序，每一步都需要兼顾时序同步与现场保护；当数码管上的时间开始精准跳动时，我深刻理解了中断驱动相较于查询方式的效率优势，也对实时系统的设计逻辑有了全新认知。实验四则将视野拓展到机间通信，通过配置 8251A 的异步通信参数，结合 8253 生成的波特率时钟，实现双机字符收发，从键盘输入到屏幕显示的完整链路，让我明白了串行通信中协议约定的重要性——哪怕只是停止位数量或校验方式的不匹配，都会导致数据传输乱码。

整个实验过程并非一帆风顺，我们曾因 8255A 控制字配置错误导致端口功能异常，也曾因 8254 计数初值计算偏差使定时不准，更在串行通信调试中因线路接触问题反复排查。但正是这些问题，让我们学会了从硬件连线到软件指令的逐环节排查。每一次问题解决后的成功，都让我们对软硬件不分家的理念有了更深刻的体会——软件指令的每一个位配置，都对应着硬件芯片的实际动作；而硬件的每一个引脚连接，都决定着软件逻辑的实现方式。

在此，我们要由衷感谢[]的指导。[]在课堂上对接口芯片原理的细致讲解，为我们奠定了理论基础；[]在实验过程中，从硬件连接规范到调试命令使用，总能在我们陷入困境时给出关键提示，帮助我们避开了许多弯路。同时，也要感谢小组四位成员的协作与支持：我们在实验前共同梳理原理、分工准备，实验中相互配合排查问题，实验后一起总结经验，每个人的思路与发现都成为团队进步的助力。

这次实验不仅让我们掌握了汇编语言编程、8255A/8254/8259/8251A 等核心芯片的应用方法，更重要的是培养了我们的系统思维与工程实践能力。这些知识与能力，将成为我们后续学习嵌入式系统、通信技术等专业课程的坚实基础，也让我们对计算机底层工作机制有了更敬畏、更深入的认知——每一个看似简单的功能背后，都是软硬件协同设计的精密逻辑。